

Advanced Topics in Artificial Intelligence

Exploring Federated Learning, Adaptive Feature Fusion, Octree Segmentation, DWT, and GAT

Group members

Name	ID
Abid Hasan	2023200000720
Joairiya Alam Sujana	2023200000719
Md. Fazley Rabby	2023200000686

Prepared for

Md. Mijanur Rahman

Assistant Professor

CSE 365.4

Artificial Intelligence

Date: 5 May 2026



Southeast University

Department of Computer Science and engineering

TABLE OF CONTENTS

Assignment Overview

This document contains five advanced topics in machine learning, signal processing, and computational geometry. Each section provides structured notes, mathematical foundations, implementation examples, and real-world applications.

No.	Assignment Topic	Page
01	Federated Learning	01 - 10
02	Adaptive Feature Fusion	11 - 19
03	Octree Segmentation	20 - 28
04	Discrete Wavelet Transform (DWT)	29 - 37
05	Graph Attention Networks (GAT)	38 - 50

Chapter 01: Federated Learning

A revolutionary paradigm that trains shared models across devices while keeping all raw data strictly local — privacy by design.

1.1 Introduction

Federated Learning (FL) is a revolutionary decentralized machine learning approach. It enables multiple devices like smartphones, IoT devices, or hospital servers to collaboratively train a shared predictive model while keeping all training data strictly local.

Instead of moving sensitive data to a centralized cloud database, Federated Learning brings the machine-learning algorithm directly to the device where the data resides. This paradigm shift ensures absolute data privacy, as raw data never leaves its origin.

1.2 Core Philosophy

The guiding principle of Federated Learning is “Send the model to where the data resides, rather than moving the data to a central location for processing.”

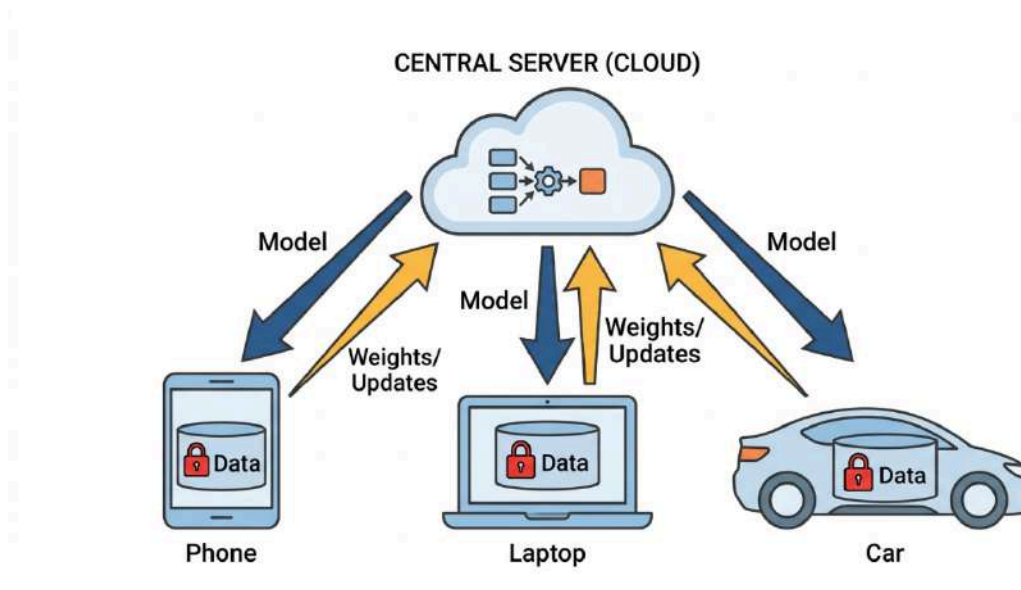


Figure 1.1 - Federated Learning Architecture: Model Flows

Down, Only Mathematical Updates Flow Up

- **Traditional ML:** Uploads all user data (photos, texts, medical records) to a central server to train a model.

- **Federated ML:** Downloads the model to the user's device, learns locally from the user's behavior, and only uploads the mathematical "lessons learned" (model weights/gradients) back to the server.

1.3 Working Process (Step-by-Step)

In large-scale production systems (like Google's network of Android phones), this process is carefully orchestrated in a synchronous loop with three main phases:

1. **Selection:** The central server selects a subset of available devices (usually a few hundred). To participate, a device must meet strict Eligibility Criteria: it must be *idle*, *charging*, and *connected to an unmetered network (WiFi)* so it doesn't drain the user's battery or cellular data.
2. **Configuration:** The server sends an "FL Plan" (training instructions) and the current "Global Model Checkpoint" to the selected devices.
3. **Local Training:** The devices perform training locally using their own private data. If the user unplugs their phone or turns off WiFi, the training aborts immediately.
4. **Reporting (Server Aggregation):** Devices upload their model updates to the server. The server waits for a specific window of time. Stragglers (slow devices) are ignored, and successful updates are combined using Federated Averaging.
5. **Global Update:** The global model is updated and the cycle repeats.

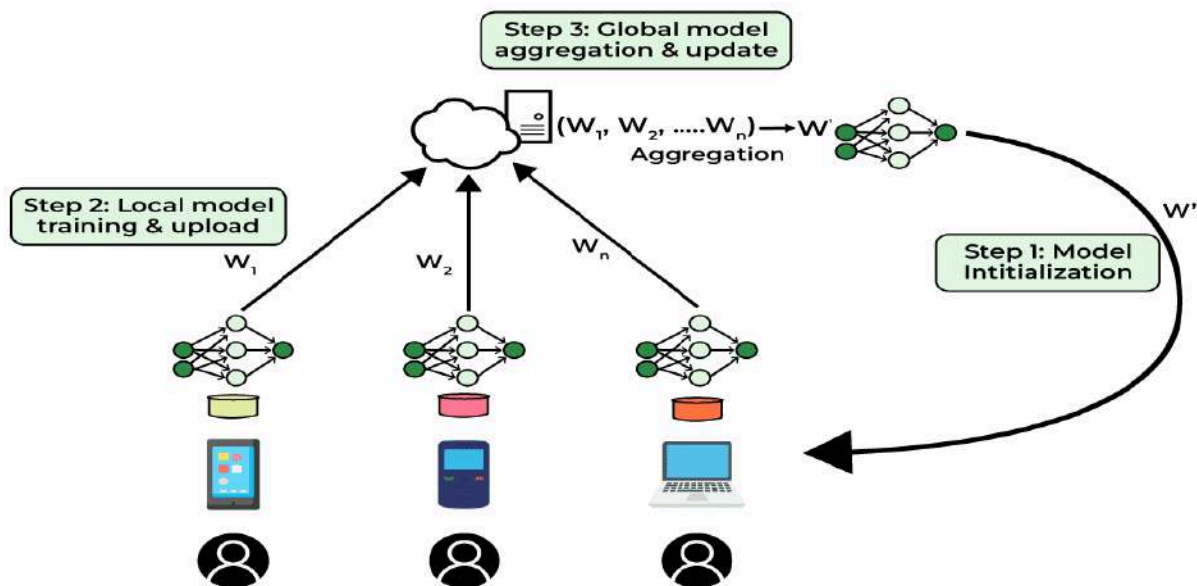


Figure 1.2 - Federated Learning Working Processes

1.4 Simple Example

Imagine 3 smartphone users (Alice, Bob, and Charlie) using a predictive text keyboard.

- During the day, they type private messages. Alice types about "football," Bob about "cooking," and Charlie about "movies."
- At night, when they are asleep, their phones are plugged into the wall and connected to WiFi.
- The central server notices they are eligible and sends them a basic predictive text model.
- Their phones learn from the day's typing locally, making the model smarter about sports, food, and movies.
- The phones send *only the mathematical updates* back to the company. The company averages them to create a universally smarter keyboard, all without ever seeing a single private text message.

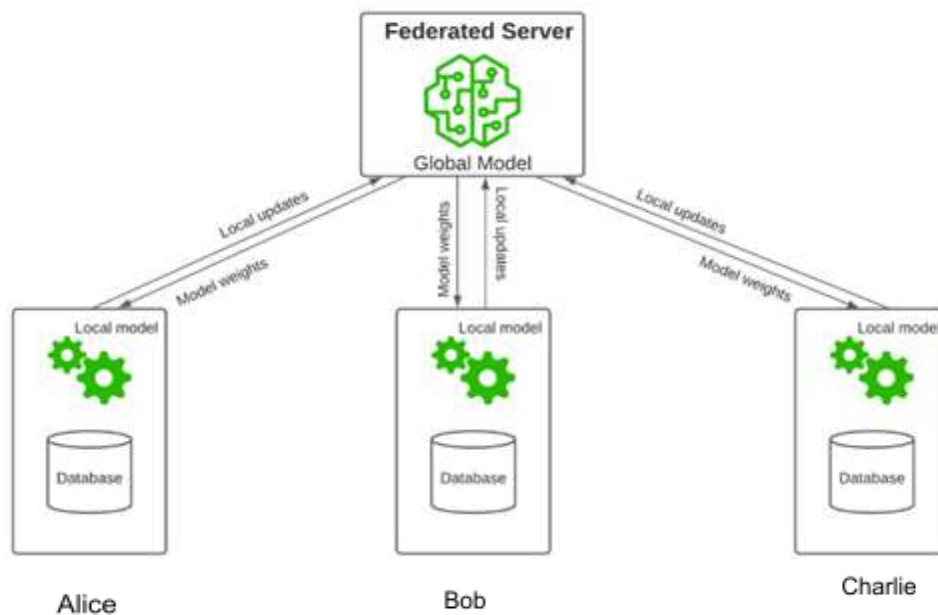


Figure 1.3 - Federated Learning Simple Example

1.5 Mathematical Formulation: Federated Averaging (FedAvg)

The most standard aggregation algorithm in FL is FedAvg. Suppose we have K participating clients. Let n_k be the number of data samples on client k , and n be the total number of data samples across all participating clients.

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$$

Where:

- $w_{(t+1)}$ → Updated global model (after aggregation on the server)
- K → Total number of clients/devices
- n_k → Number of data samples in the k -th client
- n → Total number of data samples across all clients
 $n = n_1 + n_2 + \dots + n_K$
- w^k → Model weights trained by the k -th client using its local data

1.6 Mathematical Simulation (Federated Averaging - FedAvg)

Federated Learning is a decentralized machine learning approach where multiple clients (such as mobile devices or organizations) train a model locally using their own data. Instead of sharing raw data with the server, each client only sends the updated model parameters (weights). The server then combines these local updates to create an improved global model.

One of the most widely used aggregation methods in Federated Learning is **Federated Averaging (FedAvg)**. In this method, clients that contain larger datasets contribute more strongly to the final global model because their updates are based on more training samples.

Scenario

1. Initial global model weight: $W_{initial} = 10$
2. Three clients participate with different dataset sizes:

Client	Number of Samples (n_i)	Updated Weight (w_i)
Client 1	$n_1 = 100$	$w_1 = 12$
Client 2	$n_2 = 200$	$w_2 = 15$
Client 3	$n_3 = 100$	$w_3 = 8$

Explanation of Updated Weight (w_i)

The **updated weight** represents the new model parameter value obtained after a client trains the global model using its own local dataset.

Initially, the server sends the same global model to all clients with:

$$W_{\text{initial}}=10$$

Each client then trains this model locally using its own data. After local training, the model weight changes depending on the client's dataset and learning process.

For example:

- **Client 1** trains the model using 100 samples and updates the weight from 10 to 12. Therefore, its local updated weight becomes:

$$w_1=12$$

- **Client 2** trains the model using 200 samples and updates the weight from 10 to 15:

$$w_2=15$$

- **Client 3** trains the model using 100 samples and updates the weight from 10 to 8:

$$w_3=8$$

These updated weights are then sent back to the central server. The server combines them using the FedAvg algorithm to create a new global model.

In simple terms, the updated weight shows how each client's local training modifies the original global model based on its own data.

Total number of samples:

$$N = n_1 + n_2 + n_3 = 100 + 200 + 100 = 400$$

FedAvg Formula :
$$W_{new} = \sum_{i=1}^k \frac{n_i}{N} \cdot w_i$$

Step-by-Step Calculation

1. Client 1 contribution: $\frac{100}{400} \times 12 = 0.25 \times 12 = 3$
2. Client 2 contribution: $\frac{200}{400} \times 15 = 0.50 \times 15 = 7.5$
3. Client 3 contribution: $\frac{100}{400} \times 8 = 0.25 \times 8 = 2$

Final Aggregation: $W_{new} = 3 + 7.5 + 2 = 12.5$

Final Result: $W_{new} = 12.5$

This result demonstrates how Federated Averaging gives higher importance to clients with larger datasets. **Client 2** had the largest number of samples (**200**), so its **updated weight (15)** influenced the final global model more than the other clients. This weighted aggregation helps produce a more reliable and representative global model while keeping client data private, since only model updates are shared instead of raw data.

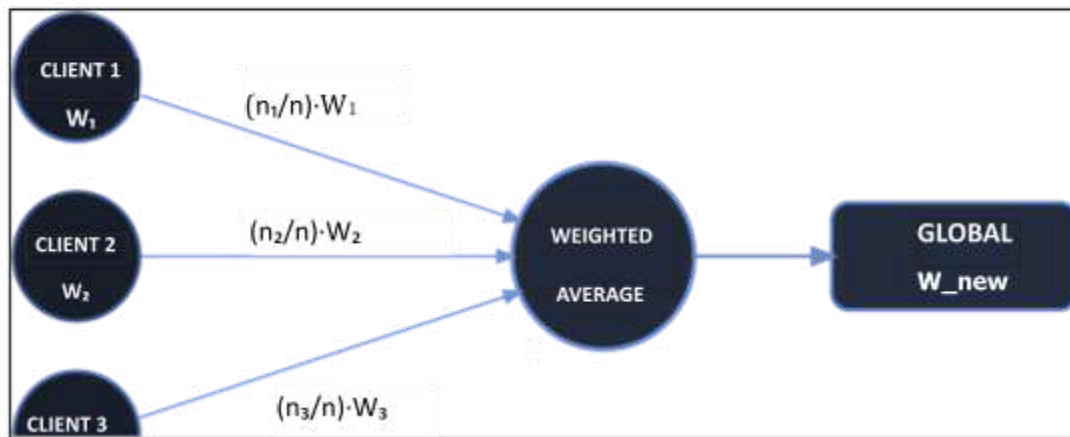


Figure 1.4 - FedAvg Aggregation: Weighted Contributions from all Clients Merge Into One Updated Global Model

1.7 Algorithm: Pseudocode for FedAvg

Pseudocode : FedAvg

Server executes:

Initialize global model weights W_0

For each round $t = 1, 2, \dots$ do:

Select a random subset of clients S_t For each client k in S_t (in parallel):

$W_k = \text{ClientUpdate}(k, W_t)$ // Send global weights to client

// Aggregate updates using weighted average

$W(t+1) = \text{sum}((n_k / n) * W_k)$ for all k in S_t

ClientUpdate(k, W):

Set local model weights $w = W$

For each local epoch:

For each batch in local dataset:

$w = w - \text{learning_rate} * \text{gradient}(\text{loss}(w, \text{batch}))$

Return w to server

1.8 Python Implementation

Note: All practical implementations for this assignment are available in my Google Colab notebook. [Click Here](#)

1.9 Security and Privacy Layer: Secure Aggregation

Even though raw data isn't shared, a hacker might still try to reverse-engineer sensitive info from the mathematical updates sent to the server. To fix this, FL uses Secure Aggregation (Multi-Party Computation).

How it works: Devices cryptographically mask their updates before sending them. The server mathematically cannot inspect an individual device's update. It can only decrypt the *final sum* of all updates combined. It's like a secret ballot where the counter only sees the final tally, not who voted for what.

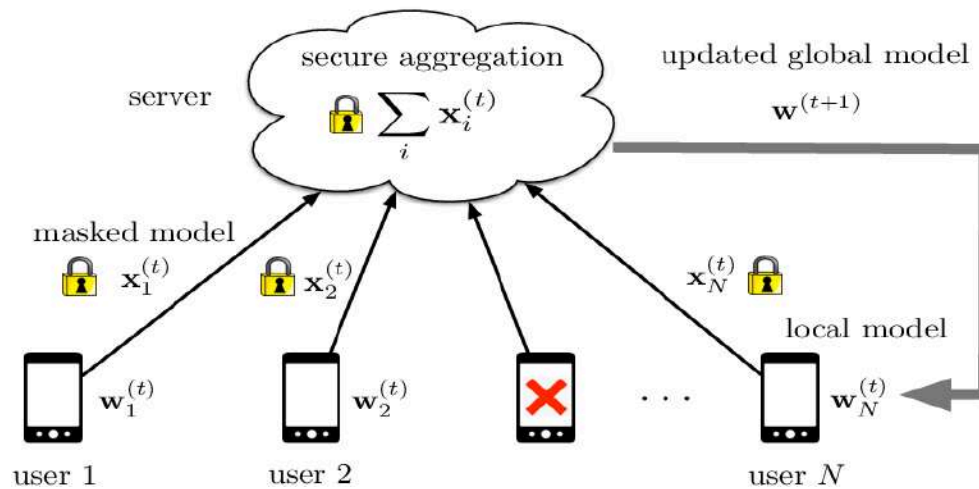


Figure 1.5 - Federated Learning : Secure Aggregation

1.10 Advantages

1. **Privacy & Data Ownership:** Raw data stays with the user, mitigating privacy leaks and complying with strict laws (like GDPR or HIPAA).
2. **Reduced Server Costs:** The heavy lifting of model training is distributed across millions of edge devices, saving massive cloud computing costs.
3. **Personalization:** Models can be easily fine-tuned on the device for the specific user's habits.

1.11 Core Challenges & Limitations

Running FL at a massive scale introduces unique problems:

1. **Expensive Communication:** AI models are huge. Uploading them takes longer than the actual training. Networks use model compression (quantization) to shrink the file sizes.

2. **Systems Heterogeneity:** Not all devices are equal. **A brand-new iPhone trains faster than a 5-year-old Android.** Dropouts are common (6-10% of devices fail to finish a round because of network loss or battery drop).
3. **Statistical Heterogeneity (Non-IID Data):** Data is not identically distributed. One user types entirely in slang, another uses formal English. This makes it hard to train a balanced global model.
4. **Diurnal Oscillations:** Device availability fluctuates massively based on time zones (e.g., millions plug their phones in at 2 AM, causing a "thundering herd" that can crash servers, requiring "pace steering" to manage connections).
5. **Bias Risks:** Because FL only trains on phones that are charging and on WiFi, the AI might unconsciously **become biased toward wealthier users or regions** with stable electricity and internet infrastructure.

LIMITATIONS OF FEDERATED LEARNING



Figure 1.6 - Federated Learning : Limitations

1.12 Real-World Applications

1. **Next-Word Prediction (e.g., Google Gboard):** FL trains the Recurrent Neural Networks (RNNs) for predictive typing. It significantly improves word recall rates without ever logging users' private messages to a central database.
2. **On-Device Item Ranking:** Personalizing search settings or app suggestions directly on a smartphone based on the user's daily habits.
3. **Healthcare Collaboration:** Multiple hospitals can collaboratively train an AI to detect diseases (like cancer in MRI scans) without illegally sharing patient medical records with outside organizations.

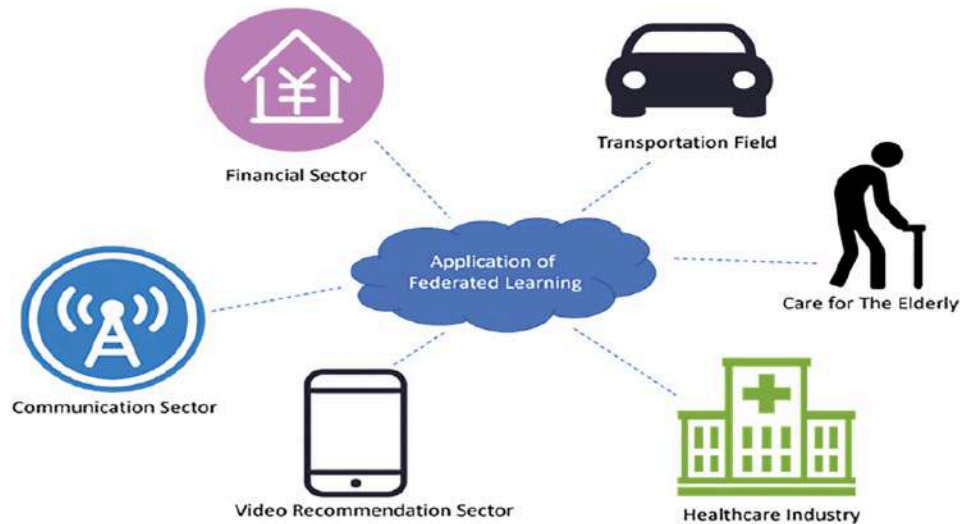


Figure 1.7 - Federated Learning : Applications

1.13 Future Directions

The field is constantly evolving. Future research focuses on Federated Analytics (gathering statistics without raw data), fighting infrastructure bias, and drastically improving data compression (like Golomb encoding) to make FL accessible even on weak 3G networks.

1.14 References for Further Reading

1. McMahan, B., et al. (2017). "Communication-Efficient Learning of Deep Networks from Decentralized Data." Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). (The foundational paper introducing FedAvg).
2. Kairouz, P., et al. (2021). "Advances and Open Problems in Federated Learning." Foundations and Trends® in Machine Learning.
3. Li, T., et al. (2020). "Federated Learning: Challenges, Methods, and Future Directions." IEEE Signal Processing Magazine.

Chapter 02: Adaptive Feature Fusion

An intelligent technique that dynamically weights and combines multiple information sources using learned attention mechanisms — not fixed rules.

2.1 Introduction

Adaptive Feature Fusion (AFF) is a dynamic and intelligent technique in deep learning where multiple information sources (**features**) are combined using weights that change based on the input data.

Unlike traditional methods that treat all incoming information as equally important, AFF intelligently selects or weights features to create a highly informative representation. This approach has become a cornerstone in advanced computer vision and multimodal learning (e.g., combining text, audio, and images).

2.2 The Core Philosophy

The core philosophy is: **"Dynamic Integration over Static Rules."** Instead of hard-coding how different data sources should be mixed, the AI model learns to blend information on the fly. It constantly asks itself: ***"Based on the current situation, which piece of data should I pay the most attention to?"***

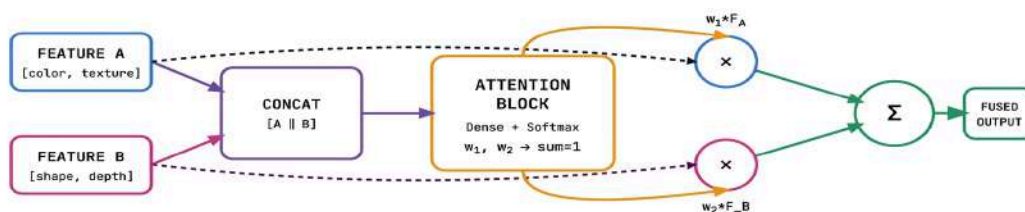


FIGURE 2.1 – AFF ARCHITECTURE: FEATURES ARE DYNAMICALLY WEIGHTED BY THE ATTENTION BLOCK BEFORE SUMMATION

2.3 Working Process (Step-by-Step)

The typical AFF process inside a neural network operates through these sequential steps:

1. Input Features Collection: The module receives features from different sources. This could be multi-scale features (different layers of a CNN) or multimodal features (e.g., camera images + LiDAR sensor data).

2. Weighting Mechanism (Attention): A small sub-network (such as a Multi-Layer Perceptron or an Attention block) analyzes these inputs to calculate dynamic gating coefficients or attention weights.

3. Dynamic Weighting: The calculated weights are applied to the input features (usually via element-wise multiplication). This step emphasizes useful information while suppressing noise or redundancy.

4. Output Fusion: The weighted features are combined (typically summed together) to produce a single, refined output feature map that is passed to the next layer.

2.4 Simple Example

Imagine you are driving a car.

- During a bright, sunny day, you rely 90% on your eyes (visual data) and maybe 10% on your hearing (audio data) to understand the road.
- But if you drive into a pitch-black tunnel with heavy fog, your vision becomes useless. Your brain dynamically adapts: suddenly, you are relying 20% on your eyes and 80% on your hearing to listen for other cars.

AFF does exactly this for AI. If a self-driving car's camera is blinded by the sun, the AFF module dynamically lowers the camera's "weight" and trusts the radar sensor more.



Figure 2.2 - Adaptive Feature Fusion: Simple Example

2.5 Mathematical Formulation:

$$\mathbf{w}_1, \mathbf{w}_2 = \mathbf{A}(\mathbf{F}_1, \mathbf{F}_2)$$

$$\mathbf{F}_{\text{fused}} = (\mathbf{w}_1 \cdot \mathbf{F}_1) + (\mathbf{w}_2 \cdot \mathbf{F}_2)$$

ADAPTIVE FUSION WITH CONTEXT-DEPENDENT WEIGHTS

Where:

- $\mathbf{F}_1$ = Feature 1 (e.g., camera features)
- $\mathbf{F}_2$ = Feature 2 (e.g., radar features)
- $\mathbf{A}(\cdot)$ = Adaptive function (computes weights based on input features)
- $\mathbf{w}_1$ = Weight for $\mathbf{F}_1$ (importance of feature 1)
- $\mathbf{w}_2$ = Weight for $\mathbf{F}_2$ (importance of feature 2)
- $\mathbf{F}_{\text{fused}}$ = Final fused feature (combined output)

2.6 Mathematical Simulation:

Let's simulate how the network calculates this for Night-Time Object Detection. In object detection systems, different types of features are extracted from an image to help identify objects accurately. Two common feature types are:

- **Color Features (F_A)** → represent color and texture information.
- **Shape Features (F_B)** → represent structural and boundary information of the object.

During daytime, both color and shape information may be useful. However, in **night-time environments**, color information becomes less reliable because of poor lighting, shadows, and low visibility. In contrast, the physical shape or outline of an object often remains more distinguishable.

To solve this problem, an **attention mechanism** dynamically assigns importance (weights) to different features based on the environment. More reliable features receive higher weights, while less useful features receive lower weights.

The Scenario:

We have two features extracted from an object:

- **F_A (Color Features) = [0.3, 0.5]**
- **F_B (Shape Features) = [0.2, 0.8]**

Since the environment is dark, the attention network decides that **shape information is more important than color information**.

Therefore, it assigns the following attention weights:

- Weight for Color Features: **$w_1=0.30$**
- Weight for Shape Features: **$w_2=0.70$**

This means the network gives only **30% importance** to color features and **70% importance** to shape features.

The Math (Dynamic Weighting):

We multiply each feature array by its weight:

- **Weighted Color = $0.30 \times [0.3, 0.5] = [0.09, 0.15]$**

- Weighted Shape = $0.70 \times [0.2, 0.8] = [0.14, 0.56]$

Final Fusion (Addition):

$$\mathbf{F}_{\text{fused}} = [0.09 + 0.14, 0.15 + 0.56] = [0.23, 0.71]$$

The final fused output shows that the second value (0.71) is much larger than the first value (0.23). This happens because the network gives more importance to shape information during the fusion process.

This demonstrates that the attention mechanism successfully adapts to the night-time environment by prioritizing the most reliable feature type. Instead of depending heavily on weak color information, the model focuses more on object shape, which improves detection accuracy in low-light conditions.

2.7 Algorithm: Pseudocode for AFF Block

Pseudocode · AFF Block

```
Function AdaptiveFeatureFusion(Feature_A, Feature_B):
    // Step 1: Concatenate features to understand the combined context
    Combined_Features = Concatenate(Feature_A, Feature_B)

    // Step 2: Pass through a small network to generate raw scores
    Raw_Scores = DenseNetwork(Combined_Features)

    // Step 3: Apply Softmax to get normalized weights (sum to 1)
    Weights_A, Weights_B = Softmax(Raw_Scores)

    // Step 4: Multiply original features by their respective weights
    Weighted_A = Feature_A * Weights_A
    Weighted_B = Feature_B * Weights_B

    // Step 5: Sum them up for the final fused output
    Fused_Output = Weighted_A + Weighted_B

    Return Fused_Output
```

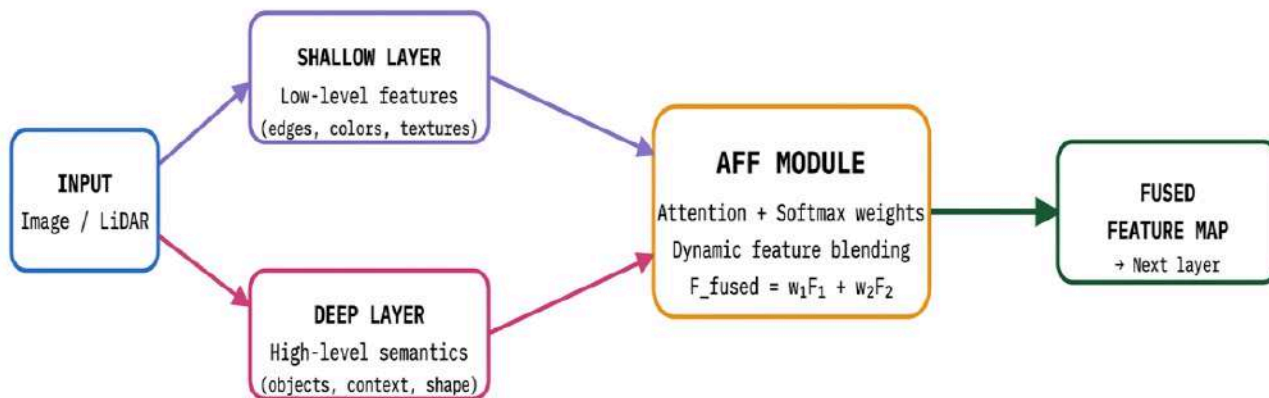


Figure 2.3 - Multi-Branch Neural Network with AFF : Shallow and Deep Features Dynamically merged

2.8 Python Implementation

Note: All practical implementations for this assignment are available in my Google Colab notebook. [Click Here](#)

2.9 Which Problems It Solves & How It Resolves Other Limitations

Traditional networks use **simple concatenation or addition**, treating every piece of data as equally reliable. The problem is, if one sensor fails or one feature is blurry, it corrupts the whole model. AFF solves this by acting as a dynamic gatekeeper—if a piece of data is noisy, misleading, or irrelevant to the current context, AFF suppresses it before it can damage the final prediction.

2.10 Advantages

1. **Enhanced Generalization:** By adapting dynamically to unseen scenarios, the model becomes highly versatile and reliable in unpredictable environments.
2. **Robustness to Noise:** It automatically down-weights noisy, corrupted, or broken sensor inputs.
3. **Feature Complementarity:** It flawlessly blends different types of data (like text and images) by finding out how they best support each other contextually.

2.11 Challenges and Limitations

1. **Computational Overhead:** Adding these mini "attention networks" increases the number of parameters the computer has to process, which can slightly slow down the model's speed.
2. **Tuning Sensitivity:** If the network is poorly trained, the attention module might "collapse." This means it gets lazy and constantly gives a weight of 1.0 to one feature and 0.0 to everything else, completely defeating the purpose of fusion.
3. **Data Hunger:** The attention mechanism requires massive, highly varied datasets to accurately learn *when* to prioritize specific features.

2.12 Real-World Applications

1. **Medical Imaging:** When analyzing MRI or X-ray scans for tumors, AFF dynamically shifts the AI's focus between "local details" (blurry edges of a tumor) and "global context" (the overall shape of the organ) depending on the clarity of the image.
2. **Autonomous Driving:** Self-driving cars use AFF to fuse LiDAR (depth mapping) and Camera (RGB visual) data. If the car drives into blinding sunlight, the model dynamically lowers the camera's weight and relies almost entirely on LiDAR to prevent crashes.
3. **Visual Question Answering (VQA):** If you show an AI an image and ask, "*What color is the dog?*", the system uses AFF to prioritize the color features of the image. If you ask, "*How many dogs are there?*", it dynamically shifts its attention to counting shapes.

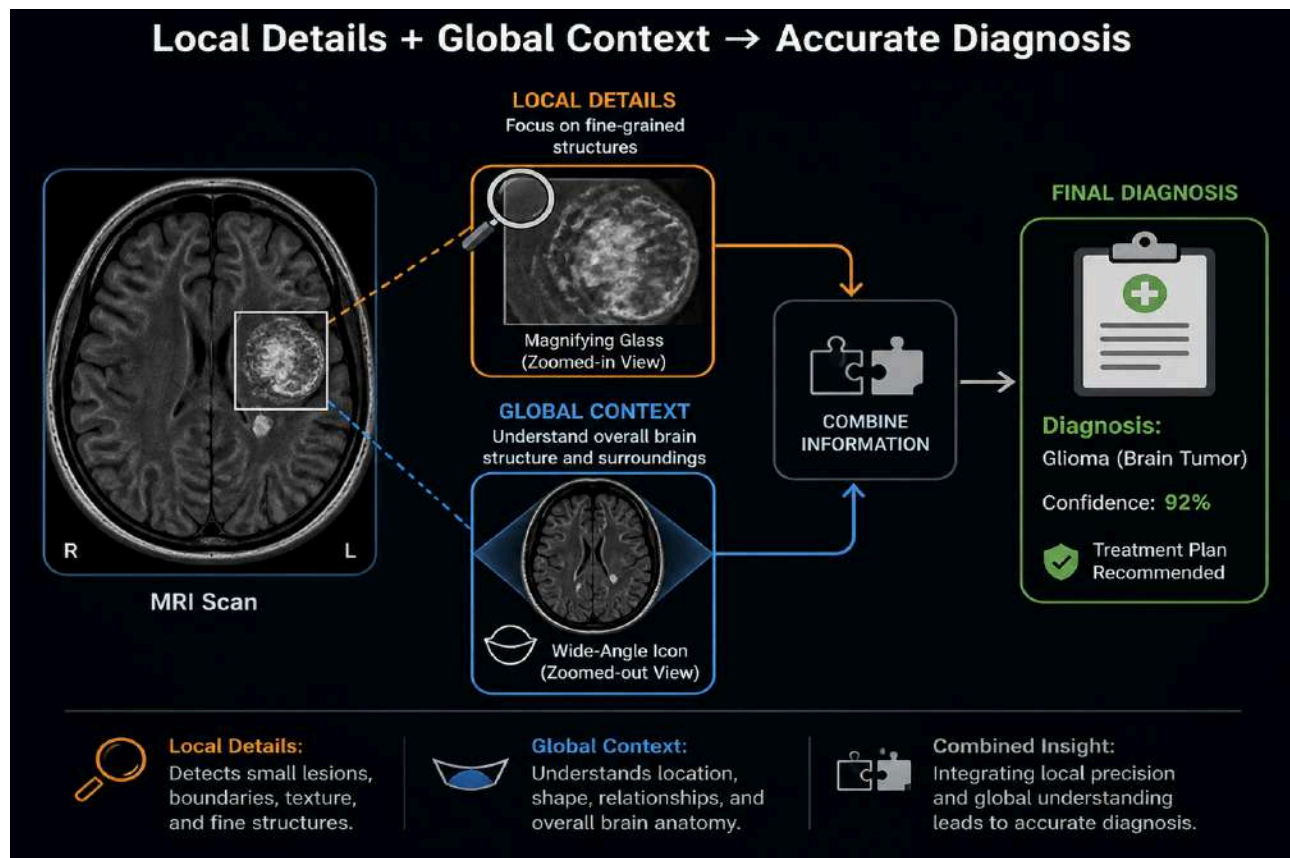


Figure 2.4 - Application of Adaptive Feature Fusion : Medical Imaging

2.13 Future Directions and Open Problems

The field of Adaptive Feature Fusion is growing rapidly. Here is what researchers are focusing on next to solve its current limitations:

1. **Ultra-Efficient Attention (Edge AI):** The mini "attention networks" used to calculate the fusion weights require extra computing power. Future research aims to make these modules incredibly lightweight so they can run on cheap, low-power devices (like smartwatches, IoT sensors, or tiny drones) without draining the battery.
2. **Handling Completely Missing Data:** Current AFF is great at lowering the weight of *bad* or *noisy* data. But what if a self-driving car's camera is completely smashed by a rock? Future AFF models need extreme robustness so they can instantly adapt when a data source goes completely offline (0% input) without the whole system crashing.
3. **Self-Taught Fusion (Unsupervised Learning):** Right now, AFF needs massive amounts of human-labeled data to learn *how* to properly weight different features. The next big step is training AFF modules to figure out what is important just by naturally observing the world, without needing humans to provide the "correct" answers.

4. **Automated Design (AutoML):** Currently, human engineers have to manually guess the best places to insert AFF "mixing boards" inside a neural network. In the future, AI will use Automated Machine Learning (AutoML) to design its own brain, automatically placing fusion blocks exactly where they will be the most effective



Figure 2.5 - Adaptive Feature Fusion: Future Directions and Open Problems

2.14 References for Further Reading

01. Dai, Y., et al. (2021). "Attentional Feature Fusion." IEEE WACV. (Foundational paper on robust AFF modules)
02. Li, X., et al. (2019). "Selective Kernel Networks." IEEE CVPR. (Dynamic selection of receptive fields)
03. Hu, J., et al. (2018). "Squeeze-and-Excitation Networks." (Pioneer paper for channel-wise attention)

Chapter 03: Octree Segmentation

A hierarchical spatial data structure that recursively divides 3D space into eight octants, enabling efficient processing of massive point clouds with $O(\log n)$ queries.

3.1 Introduction

Octree Segmentation is a highly efficient spatial partitioning technique used to organize and manage 3D data. It works by dividing a large 3D volume into smaller sections using a hierarchical tree structure where every internal node branches out into exactly eight children (octants).

3.2 The Core Philosophy

The core philosophy is: "**Recursive Spatial Subdivision.**" When dealing with 3D space, most of it is usually empty (e.g., a scan of a room has furniture, but the air in the middle is just empty space). Instead of wasting computer memory trying to map the empty air, an Octree intelligently leaves empty spaces as large, undivided blocks. It only subdivides and spends computing power on areas where actual data (points/objects) exists.

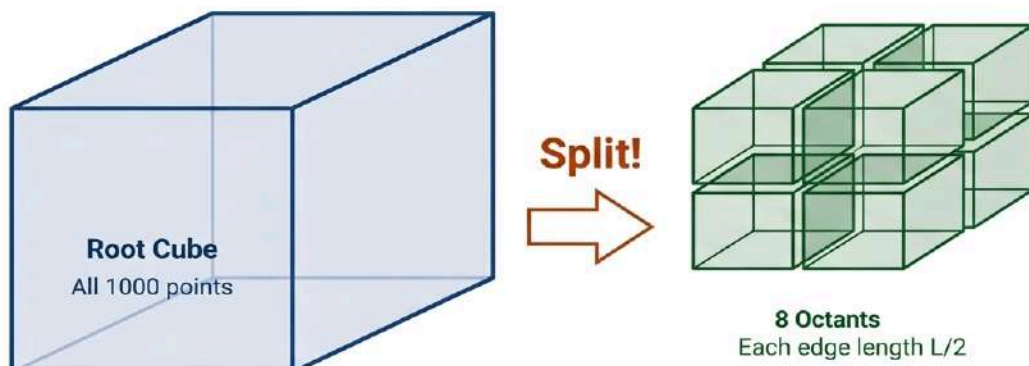


FIGURE 3.1 - OCTREE SUBDIVISION: ONE LARGE ROOT CUBE SPLITS INTO EXACTLY 8 EQUAL CHILD CUBES (OCTANTS)

3.3 Working Process (Step-by-Step)

The construction of an Octree follows a recursive top-down approach:

- 1. The Root Node:** The process begins with a single large bounding box (the "**root**" cube) that entirely encloses all 3D data points.
- 2. The Split Condition:** The algorithm evaluates a condition for example, "Does this cube contains more than N points?" or "Have we reached the maximum allowed depth?"
- 3. Subdivision (Splitting):** If the condition is met, the cube is evenly divided along its X, Y, and Z axes, resulting in **8 smaller child cubes** (octants).
- 4. Recursion:** The algorithm sorts the data points into these new octants and repeats the splitting process for each non-empty octant.
- 5. Termination:** The recursion stops when an octant meets the stopping criteria (**e.g., becomes empty, contains very few points, or reaches maximum depth**). These final undivided cubes are called "**Leaf Nodes**" or "**Voxels**".

3.4 Simple Example

Imagine you are a librarian tasked with finding a single lost book in a massive, multi-story library.

- Instead of checking every single shelf one by one (which takes forever), you divide the library into **8 large zones** (Octants).
- You check the computer to see which zone the book is in. You ignore the 7 empty zones entirely.
- Now you focus on that 1 zone and divide it into **8 smaller sections** (aisles).
- You find the aisle, ignore the rest, and divide it into **8 smaller bookcases**, and so on. By instantly eliminating massive empty areas, you find the book in a fraction of the time!

OCTREE SEGMENTATION EXAMPLE

- 1** A massive library.
We need to find
one lost book.



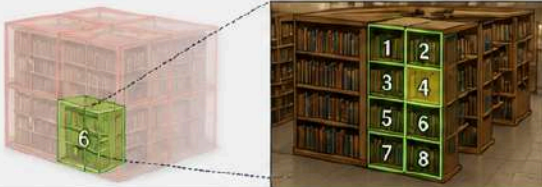
Divide the library into 8 large zones (octants).
The computer tells us the book is in **Zone 3**.

- 2** Focus only on Zone 3.
Ignore the other
7 zones.



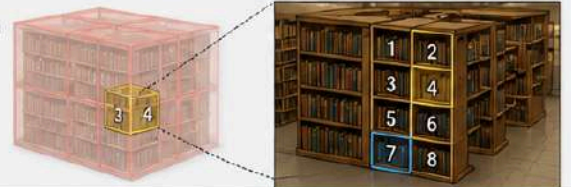
Divide Zone 3 into 8 smaller sections (aisles).
The computer tells us the book is in **Aisle 6**.

- 3** Focus only on
Aisle 6.
Ignore the
other 7
aisles.



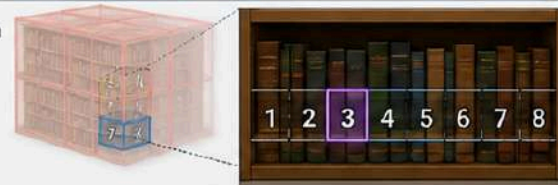
Divide Aisle 6 into 8 bookcases.
The computer tells us the book is in **Bookcase 4**.

- 4** Focus only on
Bookcase 4.
Ignore the
other 7
bookcases.



Divide Bookcase 4 into 8 shelves.
The computer tells us the book is on **Shelf 7**.

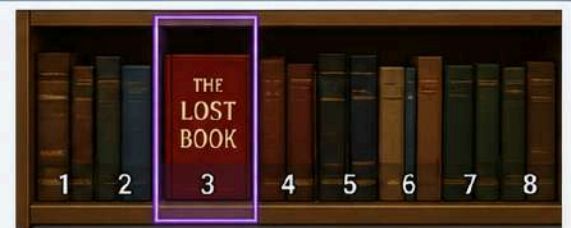
- 5** Focus only on
Shelf 7.
Ignore the
other 7
shelves.



Divide Shelf 7 into 8 slot positions.
The computer tells us the book is in **Slot 3**.

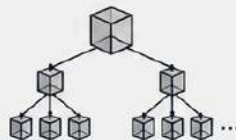
- 6 Found!**

The book is
in Slot 3.
We found it
in just a few
steps!



OCTREE BENEFIT

By recursively dividing space into 8 parts and focusing only on the relevant part at each step, we eliminate massive empty areas and find the target extremely efficiently.



KEY IDEA

Each step reduces the search space to 1/8th of the previous one.

Very fast: $O(\log_8 N)$ steps!

Figure 3.2 -Octree Segmentation: Simple example of Library

3.5 Mathematical Formulation

Let a 3D bounding box be defined by center $C = (x_c, y_c, z_c)$ and side length L . When split:

$$L_{\text{child}} = L / 2$$

$$C_{\text{child}} = (x_c \pm L/4, y_c \pm L/4, z_c \pm L/4)$$

8 new child centers generated by $\pm$ combinations on all 3 axes

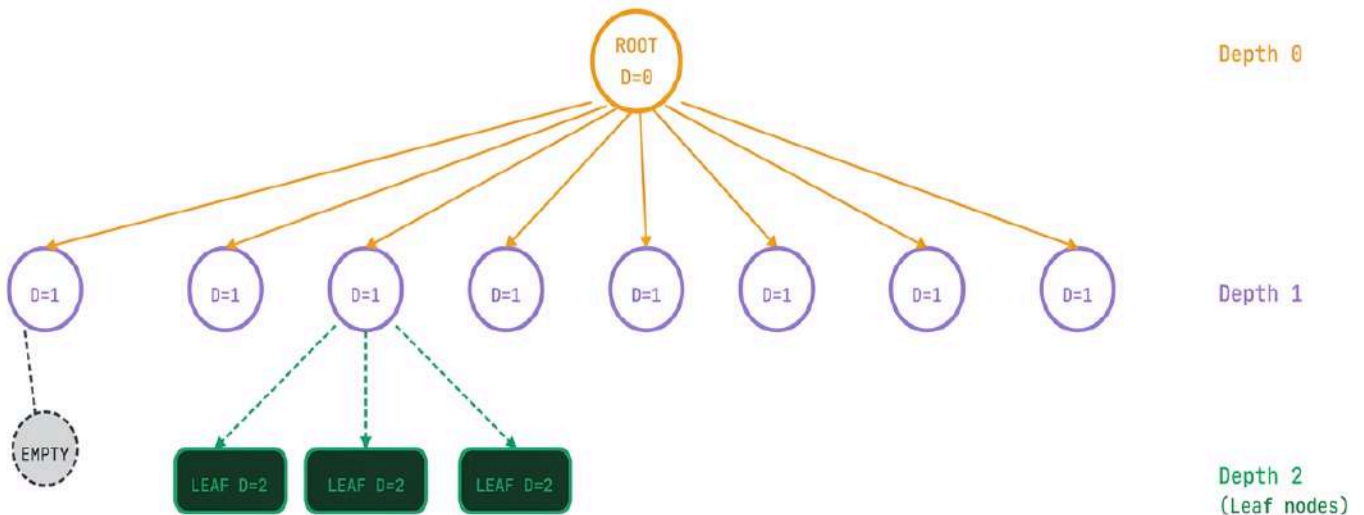


Figure 3.3 -Octree Hierarchy: Root Branches Into 8, Then 8 Again Recursively. Empty Octants Are Not Expended (Saves Memory)

3.6 Mathematical Simulation

Let's simulate how the math perfectly calculates the new smaller cubes when an Octree splits.

The Scenario:

Assume we have a large 3D bounding box (the Parent Cube) with:

- **Center coordinates (C):** (X=0, Y=0, Z=0)
- **Side Length (L):** 100 meters

The cube contains too many points, so we must split it into 8 child cubes.

The Math (Subdivision):

1. **New Size:** The side length of every child cube is exactly half of the parent.

$$L_{\text{child}} = \frac{L}{2} = \frac{100}{2} = 50 \text{ meters.}$$

2. **New Centers:** To find the center coordinates of the 8 new child cubes, we shift the parent's center by exactly one-quarter of the original length ($\frac{L}{4} = 25$) in all possible positive and negative directions for X, Y, and Z.

$$C_{\text{child}} = (X_c \pm \frac{L}{4}, Y_c \pm \frac{L}{4}, Z_c \pm \frac{L}{4})$$

Let's calculate a few of the 8 new child centers:

- **Child 1 (Top-Right-Front):** $(0 + 25, 0 + 25, 0 + 25) = (25, 25, 25)$
- **Child 2 (Bottom-Left-Back):** $(0 - 25, 0 - 25, 0 - 25) = (-25, -25, -25)$
- **Child 3 (Top-Left-Front):** $(0 - 25, 0 + 25, 0 + 25) = (-25, 25, 25)$
(...and so on for all 8 combinations).

3.7 Pseudocode for Octree Construction

Pseudocode · Octree Build

```
Function BuildOctree(Cube, Points, Depth, Max_Depth, Max_Points):
```

```
    // Base Case: Stop if max depth reached or few points
```

```
    If Depth == Max_Depth OR Points.Count <= Max_Points:
```

```
        Return LeafNode(Cube, Points)
```

```
    SubCubes = DivideInto8(Cube)
```

```
    Children = []
```

```
    For each sub_cube in SubCubes:
```

```
        PointsInSubCube = GetPointsInside(Points, sub_cube)
```

```
// Only branch if the sub-cube has points (saves memory!)
If PointsInSubCube.Count > 0:
    ChildNode = BuildOctree(sub_cube, PointsInSubCube, Depth + 1, Max_Depth,
Max_Points)
    Children.Append(ChildNode)

Return InternalNode(Cube, Children)
```

3.8 Python Implementation

Note: All practical implementations for this assignment are available in my Google Colab notebook. [Click Here](#)

3.9 Which Problems It Solves & How It Resolves Other Limitations

Traditional 3D representations (like dense 3D Voxel Grids) have a fatal flaw: Cubic Complexity ($O(N^3)$). If you double the resolution of a 3D grid, the memory required multiplies by 8. Trying to map an entire city in high resolution using a dense grid would instantly crash any computer because it wastes RAM trying to store empty air. Octrees solve this by using a Sparse Representation. By instantly stopping subdivision in empty areas, memory usage drops drastically, allowing computers to process massive environments.

3.10 Advantages

1. **Massive Memory Efficiency:** Memory is only allocated where actual physical data exists, completely ignoring empty space.
2. **Lightning-Fast Querying:** Searching for a specific point or collision takes logarithmic time $O(\log n)$ because the algorithm can safely skip massive empty areas of the tree.
3. **Extreme Scalability:** It can handle gigantic datasets, like billions of LiDAR points from a city-wide aerial scan, without crashing the system.

THE PROBLEM WITH TRADITIONAL 3D REPRESENTATIONS

! CUBIC COMPLEXITY ($O(N^3)$)

In a dense 3D voxel grid, memory grows with the cube of the resolution.

Resolution	Memory
N	= N^3
2N	= $8N^3$ (8x)
4N	= $64N^3$ (64x)
8N	= $512N^3$ (512x)

DENSE 3D VOXEL GRID

EXTREMELY HIGH MEMORY USAGE

SLOWS DOWN COMPUTATION

WASTES MEMORY ON EMPTY SPACE (AIR)

! Trying to map an entire city in high resolution using a dense grid would instantly crash any computer due to massive memory usage.

THE SOLUTION: OCTREES (SPARSE REPRESENTATION)

✓ SPARSE & EFFICIENT

Octrees adaptively subdivide space only where there is data. Empty regions are not subdivided.

WHY OCTREES ARE BETTER

- ✓ Memory usage drops drastically
- ✓ Ignores empty space (air)
- ✓ Handles massive environments
- ✓ Enables real-time processing

OCTREE REPRESENTATION

LOW MEMORY USAGE

FASTER COMPUTATION

STORES ONLY USEFUL DATA

Empty space (not stored)

Occupied space (subdivided)

COMPARISON

Example: Mapping a City (High Resolution)

DENSE VOXEL GRID

Memory Usage: ~1000 GB+

Wasted on Empty Space: ~95%+

Feasible: **X No**

OCTREE

Memory Usage: ~20-50 GB

Wasted on Empty Space: <10%+

Feasible: **✓ Yes**

KEY TAKEAWAY

By instantly stopping subdivision in empty areas, octrees reduce memory from $O(N^3)$ to $\sim O(N \log N)$ in practice, making large-scale 3D mapping possible.

Figure 3.4 - Advantages of Octree Segmentation

3.11. Challenges & Limitations

- Blocky Edges (Voxelization Effect):** At lower depths/resolutions, the boundaries of objects look like jagged zig-zags (like Minecraft blocks) rather than smooth curves.
- Dynamic Data Bottlenecks:** Octrees are incredibly fast for *static* scenes (things that don't move). However, if an object moves (like a character running in a video game), the entire tree structure often has to be constantly recalculated and rebuilt, which is computationally expensive.
- Implementation Complexity:** Building the tree efficiently in lower-level programming languages (like C++) requires highly complex memory allocation and pointer management to prevent memory leaks.

3.12 Real-World Applications

1. **Autonomous Vehicles (LiDAR):** Self-driving cars constantly blast lasers (LiDAR) to map the road. They use Octrees to instantly segment and identify drivable roads, pedestrians, and obstacles from millions of incoming laser points in real-time.
2. **Computer Graphics & Video Games:** Used heavily for "Frustum Culling" and fast collision detection. When you shoot a gun in a game, the engine uses an Octree to quickly calculate if the bullet hit a wall without having to test the bullet against every single polygon in the entire city.
3. **Medical Imaging (MRI/CT Scans):** Efficiently rendering and storing ultra-high-resolution 3D medical scans. Doctors can load a full-body scan efficiently and use the Octree structure to instantly zoom in and render specific dense tissues (like a tumor) in maximum detail.

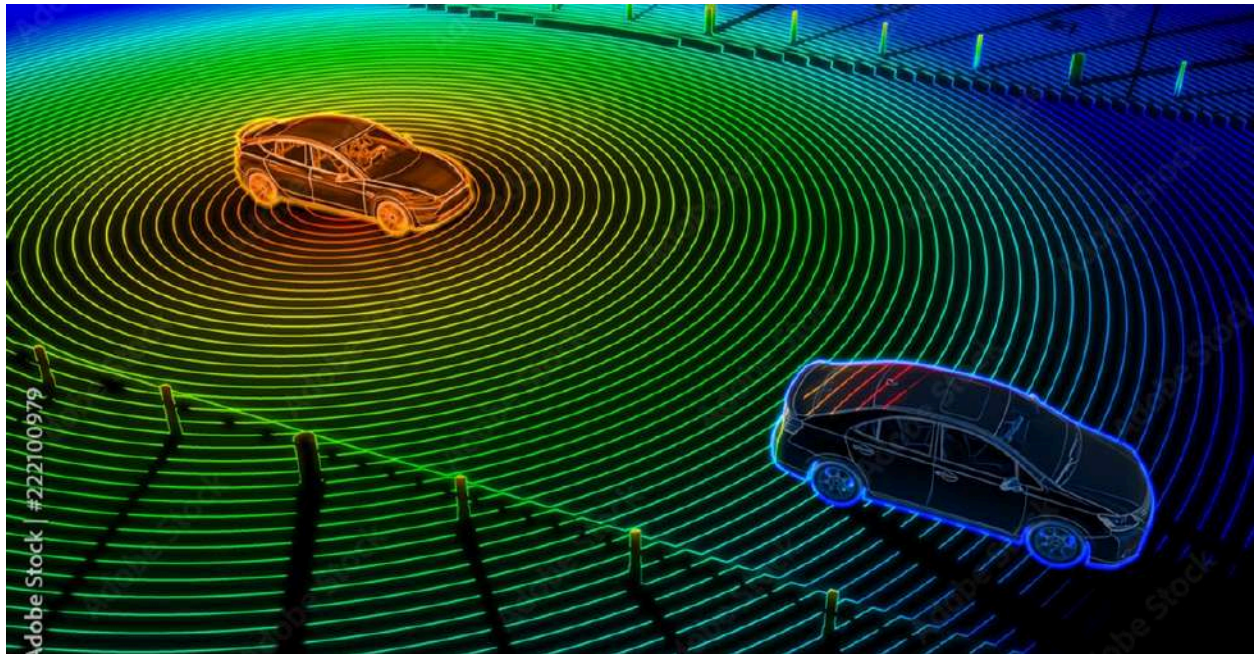


Figure 3.5 - Octree Segmentation : Real World Application - Autonomous Vehicles (LiDAR)

3.13 Future Directions and Open Problems

As 3D data from self-driving cars, drones, and virtual reality continues to explode in size, researchers are focusing on solving the remaining bottlenecks of Octree Segmentation:

1. **Native Octree Deep Learning (O-CNNs & OctFormers):** Standard AI models (like Convolutional Neural Networks) were built for flat 2D images. Researchers are actively inventing new AI architectures that can natively "think" and process data directly inside

an Octree structure. This would allow AI to understand 3D environments infinitely faster by completely skipping the empty spaces during training.

2. **Real-Time Dynamic Updating:** Currently, if a character runs through a 3D forest, the computer often has to delete and rebuild the entire Octree because the scene changed. A major open problem is developing "Dynamic Octrees" that can intelligently update *only the specific branch* of the tree where movement occurred, without touching the rest of the static environment.
3. **Extreme 3D Data Compression:** A single self-driving car can generate terabytes of LiDAR point cloud data in just one day. Future research (like "OctSqueeze") is exploring how to use the Octree structure as a massive compression tool—essentially creating a highly efficient "ZIP file" format specifically for 3D spatial data so it can be streamed over 5G networks.
4. **Hybrid Rendering (Octrees + AI Graphics):** Cutting-edge AI graphics, like Neural Radiance Fields (NeRFs) and 3D Gaussian Splatting, are incredibly realistic but very slow to render. Researchers are trying to combine Octrees with these AI models to instantly eliminate empty space calculations, making photorealistic AI-generated 3D worlds playable in real-time video games.

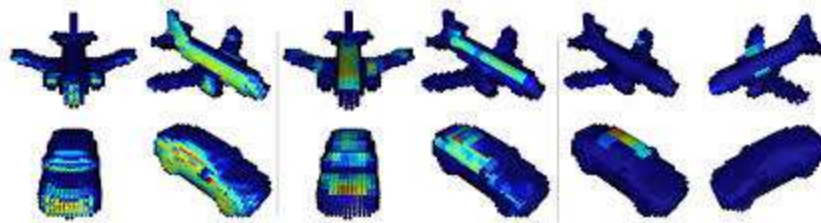


Figure 3.6 - Octree Segmentation : O-CNN: Octree-based Convolutional Neural Networks for 3D Shape Analysis

3.14 References for Further Reading

01. Meagher, D. (1982). "Geometric modeling using octree encoding." Computer Graphics and Image Processing. (The foundational paper)
02. Samet, H. (1990). "The Design and Analysis of Spatial Data Structures." Addison-Wesley.
03. Elseberg, J., et al. (2013). "One billion points in the cloud." ISPRS Journal of Photogrammetry and Remote Sensing.

Chapter 04: Discrete Wavelet Transform

A powerful mathematical tool for time-frequency signal analysis that captures both when and what frequency anomalies occur, the foundation of JPEG 2000 compression.

4.1 Introduction

The Discrete Wavelet Transform (DWT) is a powerful mathematical tool used to analyze digital signals. It quantifies how much energy is contained in specific frequency bands at particular moments in time. Unlike the Fourier Transform, DWT provides both **time and frequency** information simultaneously.

4.2 The Core Philosophy

The core philosophy of DWT is: "**Time-Frequency Localization.**" To put it simply: The standard Fast Fourier Transform (FFT) tells you *what* frequencies exist in a signal, but the DWT tells you exactly *when* those frequencies occur.

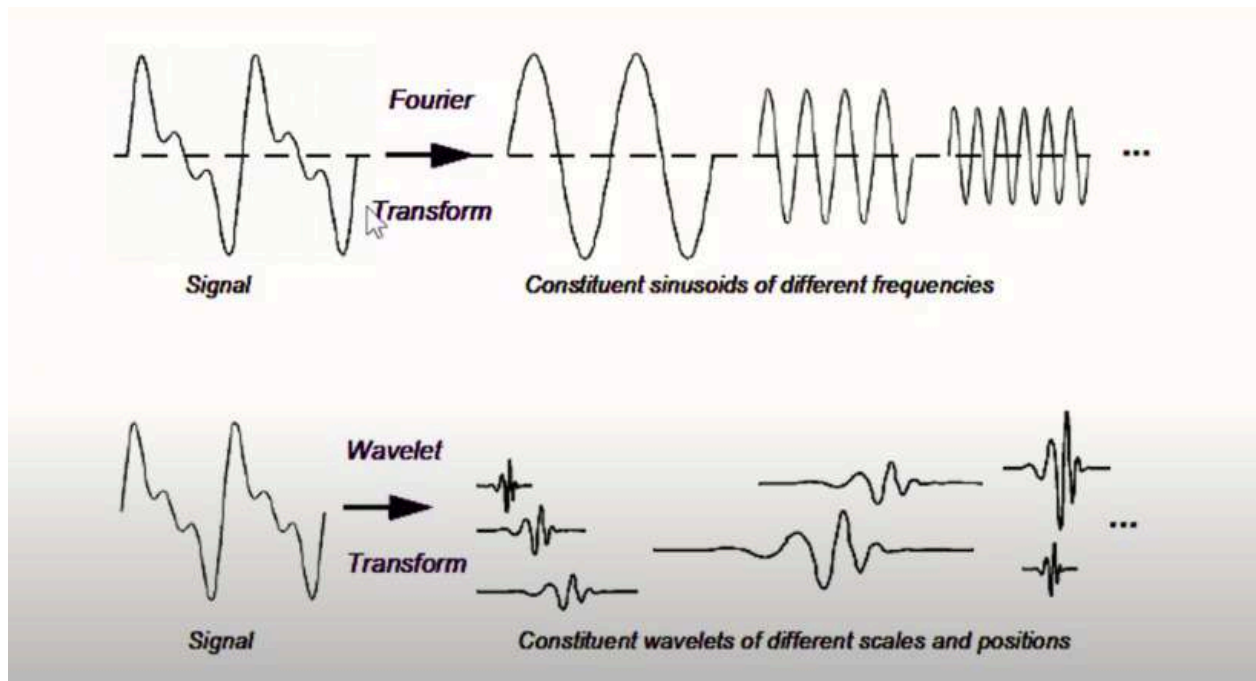


Figure 4.1 - Fourier Transform Vs Wavelet Transform

4.3 Working Process (The Pyramid Algorithm)

DWT processes a signal through a cascading step-by-step loop known as the Pyramid Algorithm (or Mallat's Algorithm):

1. **Input:** The raw 1D digital data is fed into the system.
2. **Low-Pass Filter (cA):** The signal passes through a low-pass filter to extract "coarse" information. This represents the general trend or average of the data. These are called **Approximation Coefficients (cA)**.
3. **High-Pass Filter (cD):** The signal simultaneously passes through a high-pass filter to capture "fine" details—like sudden spikes or sharp edges. These are called **Detail Coefficients (cD)**.
4. **Downsampling ($\downarrow 2$):** Because we have split the frequency band in half, we now have twice as much data. To keep the data size constant, every second sample is safely discarded (downsampled).
5. **Multi-Level Decomposition :** Steps 1–4 are repeated recursively *only* on the Approximation (cA) output, creating a cascading tree of deeper and deeper analysis.

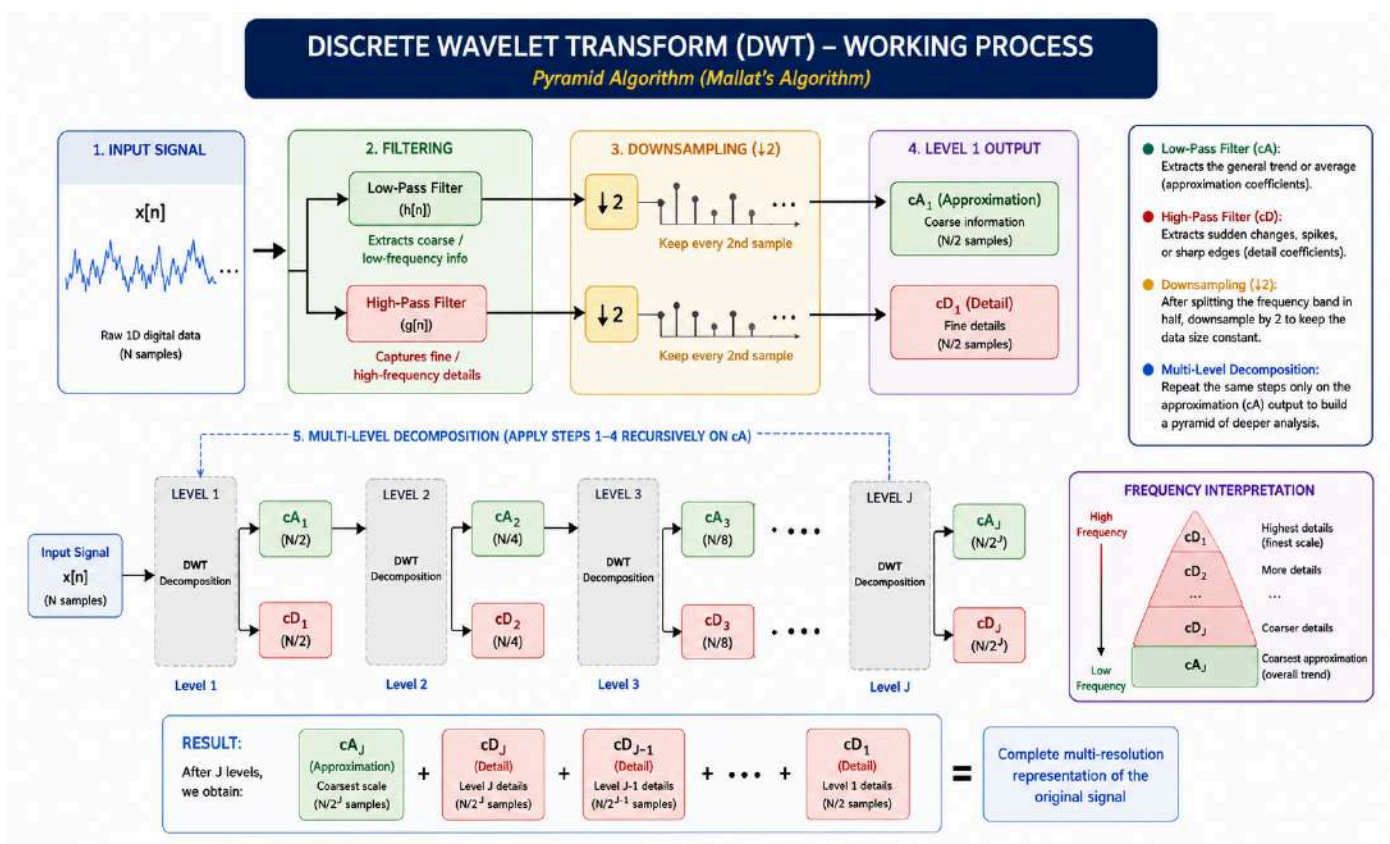


Figure 4.2 - DWT : Working Process

4.4 Simple Example (JPEG 2000 Image Compression)

Imagine you are downloading a high-resolution photograph of a perfectly smooth blue sky over a sharp, rocky mountain. You need to compress the file size so it downloads faster.

- **Standard JPEG (Old Method):** This method brutally chops the image into tiny 8x8 square blocks and compresses each block individually. When highly compressed, the smooth blue sky gets ruined, looking like a mosaic of ugly, pixelated squares (blocky artifacts).
- **JPEG 2000 (Using Discrete Wavelet Transform):** Instead of chopping the image into blocks, the DWT analyzes the *entire* picture at once. It intelligently separates the "**broad strokes**" (the smooth blue sky and general mountain shape) from the "**fine details**" (the sharp cracks in the rocks).



Jpeg



Jpeg 2000

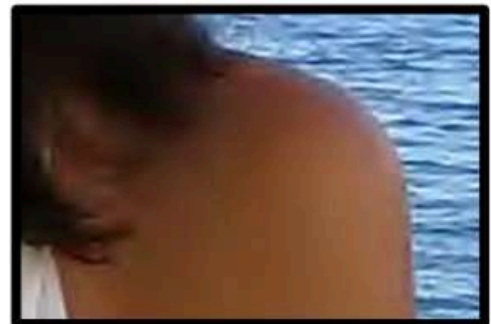


Figure 4.3 - DWT : Simple Example of Image Compression (Jpeg 2000)

4.5 Mathematical Formulation (Haar Wavelet)

$$cA_i = (x_{2i-1} + x_{2i}) / \sqrt{2}$$

$$cD_i = (x_{2i-1} - x_{2i}) / \sqrt{2}$$

Haar Wavelet — simplest wavelet basis. Scaling by $\frac{1}{\sqrt{2}}$ preserves (conserves) the signal energy.

Where:

- cA_i denotes the *approximation coefficient*, representing the low-frequency (coarse or average) component of the signal.
- cD_i denotes the *detail coefficient*, representing the high-frequency (fine detail) component of the signal.
- x_{2i-1}, x_{2i} are two consecutive samples of the input signal.
- The factor $\frac{1}{\sqrt{2}}$ is used for **normalization**, ensuring that the total energy of the signal remains preserved after transformation.

4.6 Mathematical Simulation

Let's simulate a 1-Level DWT using the **Haar Wavelet** (the simplest wavelet basis). The Haar wavelet groups data points into pairs, finds their average (Approximation), and finds their difference (Detail). It multiplies by $\frac{1}{\sqrt{2}}$ to conserve signal energy.

The Scenario:

Let our raw input signal be an array of 4 numbers:

$X = [1, 2, 3, 4]$

The Math (Haar DWT):

We process the numbers in pairs: (1, 2) and (3, 4).

1. Approximation Coefficients (cA): *(The Averages)*

- Pair 1: $\frac{1+2}{\sqrt{2}} = \frac{3}{\sqrt{2}} \approx 2.121$
- Pair 2: $\frac{3+4}{\sqrt{2}} = \frac{7}{\sqrt{2}} \approx 4.950$
- Final cA = [2.121, 4.950]

2. Detail Coefficients (cD): *(The Differences)*

- Pair 1: $\frac{1-2}{\sqrt{2}} = \frac{-1}{\sqrt{2}} \approx -0.707$
- Pair 2: $\frac{3-4}{\sqrt{2}} = \frac{-1}{\sqrt{2}} \approx -0.707$
- Final cD = [-0.707, -0.707]

This simulation shows that the Discrete Wavelet Transform (DWT) separates the original signal into two parts: Approximation coefficients (cA) that contain the main trend information and Detail coefficients (cD) that contain the small changes or differences in the signal.

Here, cA=[2.121, 4.950] stores the overall structure of the signal, while cD=[-0.707, -0.707] stores the detailed variations between adjacent values. The Haar Wavelet also preserves signal energy using the factor $\frac{1}{\sqrt{2}}$.

An important advantage of DWT is that the original signal $X=[1,2,3,4]$ can be reconstructed again from the approximation and detail coefficients without losing important information. Therefore, DWT is very useful for signal compression, denoising, and image processing applications.

4.7 Algorithm: Pseudocode for 1D Haar DWT

Here is how the algorithm works logically without complex libraries:

Pseudocode for 1D Haar DWT

```
Function Haar_DWT_1D(Signal):  
  
Length = size(Signal)  
cA = empty_list() // Approximation  
cD = empty_list() // Detail  
  
// Process pairs of numbers (stride of 2)  
  
For i from 0 to Length-1 step 2:  
    val1 = Signal[i]  
    val2 = Signal[i+1]  
  
    // Calculate and apply energy scaling factor  
  
    approx = (val1 + val2) / sqrt(2)  
    detail = (val1 - val2) / sqrt(2)  
    cA.append(approx)  
    cD.append(detail)  
Return cA, cD
```

4.8 Python Implementation (PyWavelets)

Note: All practical implementations for this assignment are available in my Google Colab notebook. [Click Here](#)

4.9 Which Problems It Solves & How It Resolves Other Limitations

The traditional Fast Fourier Transform (FFT) struggles with "non-stationary" signals (signals where the frequencies change over time, like human speech or a heartbeat). Because FFT analyzes the *entire* signal at once, the time information is completely lost. DWT resolves this by analyzing the signal using short, localized mathematical waves (wavelets). This allows it to pinpoint the exact moment an anomaly (like a sudden glitch or a heart murmur) occurs.

FFT vs DWT: Capturing Changes Over Time

Why DWT is better for non-stationary signals

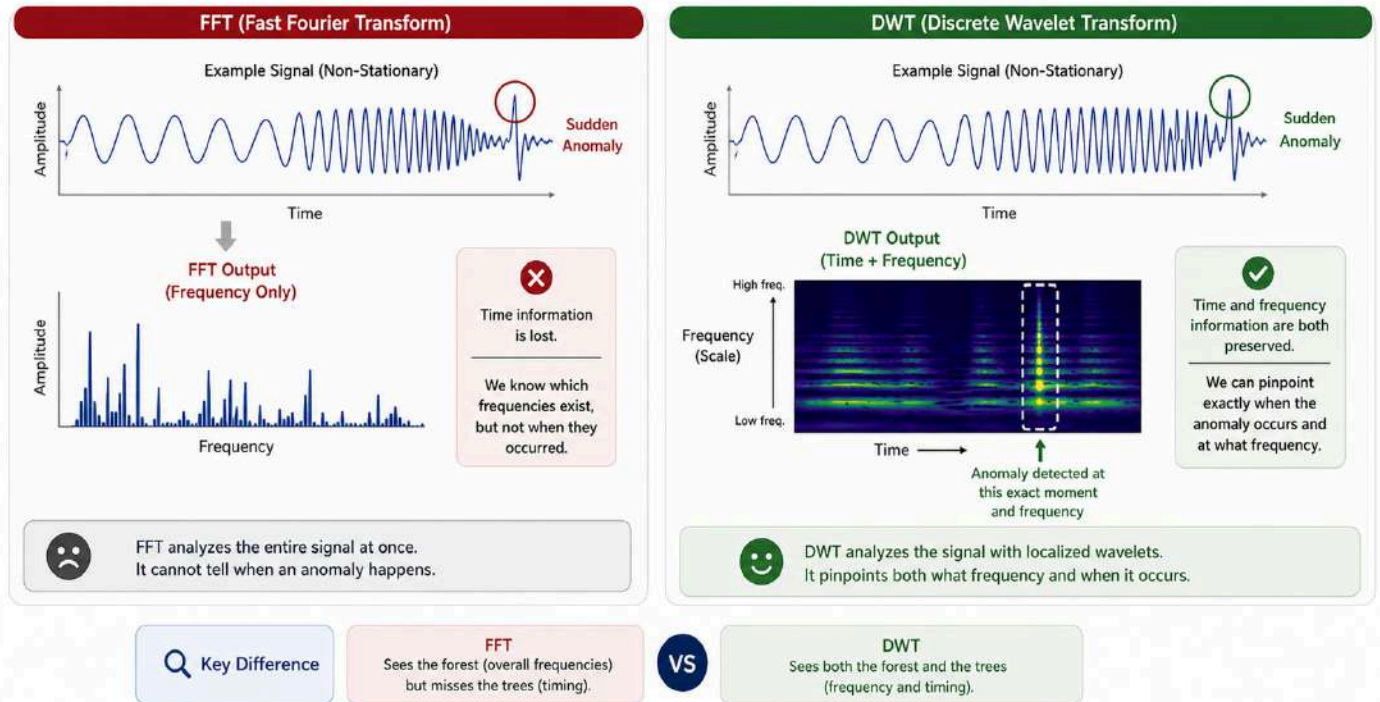


Figure 4.4 - DWT : How DWT Resolves Limitations of FFT

4.10 Advantages

1. **Perfect Localization:** Tells us exactly *where* a sudden frequency change occurs in time.
2. **Extreme Sparsity:** In natural images or audio, most of the "Detail" coefficients end up being exactly zero. This makes DWT perfect for high-ratio data compression.
3. **Computational Speed:** The pyramid filter bank implementation takes only $O(N)$ time, which is actually faster than FFT's $O(N \log N)$.

4.11 Challenges / Limitations

1. **Shift Sensitivity:** The standard DWT is highly sensitive to time alignment. If you shift the input signal by just a single tiny sample, it can drastically alter all the resulting coefficients.

2. **Power of Two Requirement:** Basic DWT algorithms require the length of the signal to be exactly a power of two (2, 4, 8, 16, 1024...). If it isn't, you must artificially pad the data with zeros, which can introduce errors.
3. **Mathematical Complexity:** Choosing the correct "Mother Wavelet" (Haar, Daubechies, Symlet) requires a deep mathematical understanding of the specific signal you are analyzing.

4.12 Real-World Applications

1. **Data Compression (JPEG 2000):** DWT is the core engine behind the JPEG 2000 image standard. By converting an image into wavelets and discarding the zero-value details, it offers incredible compression without the ugly "blocky" artifacts seen in old standard JPEGs.
2. **Biomedical Engineering:** Medical devices use DWT to categorize and analyze sharp, sudden anomalies in ECG (heart monitoring) or EEG (brainwave) data, immediately flagging irregular heartbeats.
3. **Signal De-noising:** It is heavily used to remove background hiss from old audio recordings or grain from digital photos. Engineers simply force the very small Detail Coefficients (noise) to zero, and perfectly reconstruct the clean signal.

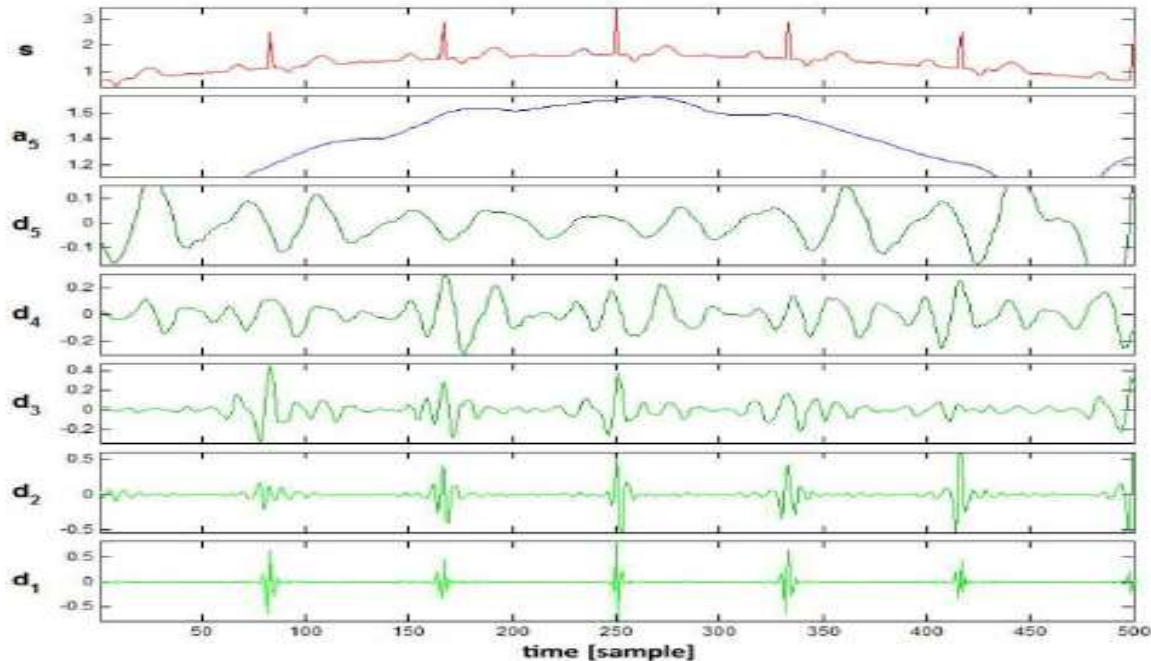


Figure 4.5 - DWT : Real-World Applications in heart monitoring

4.13. Future Directions

1. **Wavelet Neural Networks (WaveC-Nets):** Deep learning models traditionally use "pooling" layers that throw away a lot of structural data. Researchers are now **embedding DWT directly into Neural Networks** to replace pooling, helping AI preserve fine structural details while resisting noise.
2. **Shift-Invariant Transforms:** Solving the "shift sensitivity" problem by developing complex Dual-Tree Wavelet Transforms that remain stable no matter how the input signal shifts in time.

4.14 References for Further Reading

1. Mallat, S. G. (1989). "A Theory for Multiresolution Signal Decomposition." IEEE TPAMI. (Definitive Filter Bank paper)
2. Daubechies, I. (1992). "Ten Lectures on Wavelets." SIAM. (The ultimate wavelet mathematics textbook)
3. Lee, T. S. (1996). "Image Representation Using 2D Gabor Wavelets." IEEE TPAMI.

Chapter 05: Graph Attention Networks

A powerful graph neural network architecture that dynamically assigns importance weights to each neighbor node — enabling interpretable, attention-driven representation learning on graphs.

5.1 Introduction

Graph Attention Networks (GAT) represent a highly powerful and advanced class of neural architectures within the broader field of Graph Neural Networks (GNNs). They are designed specifically to operate on graph-structured data, such as social networks, molecular structures, and citation networks. The defining feature of a GAT is its use of an "Attention Mechanism," which allows a node to dynamically decide which of its neighbors' information is the most critical and relevant for its own representation.

5.2 The Core Philosophy

Core Principle — "Dynamic Attention over Static Aggregation."

- *Standard GNNs (e.g., GCNs): Typically aggregate information from all neighboring nodes with equal importance or based on a fixed structural rule.*
- *GATs: Introduce the ability to assign different importance (weights) to different neighbors. This means a node can focus heavily on neighbors that hold highly relevant features while effectively ignoring noise from less important ones.*

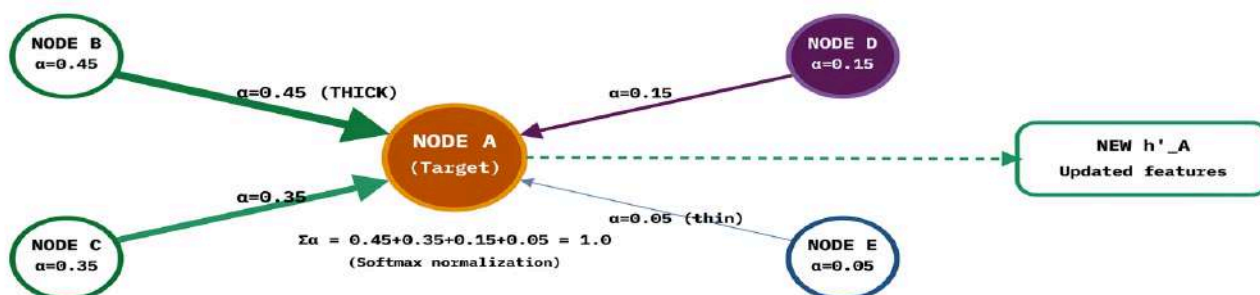
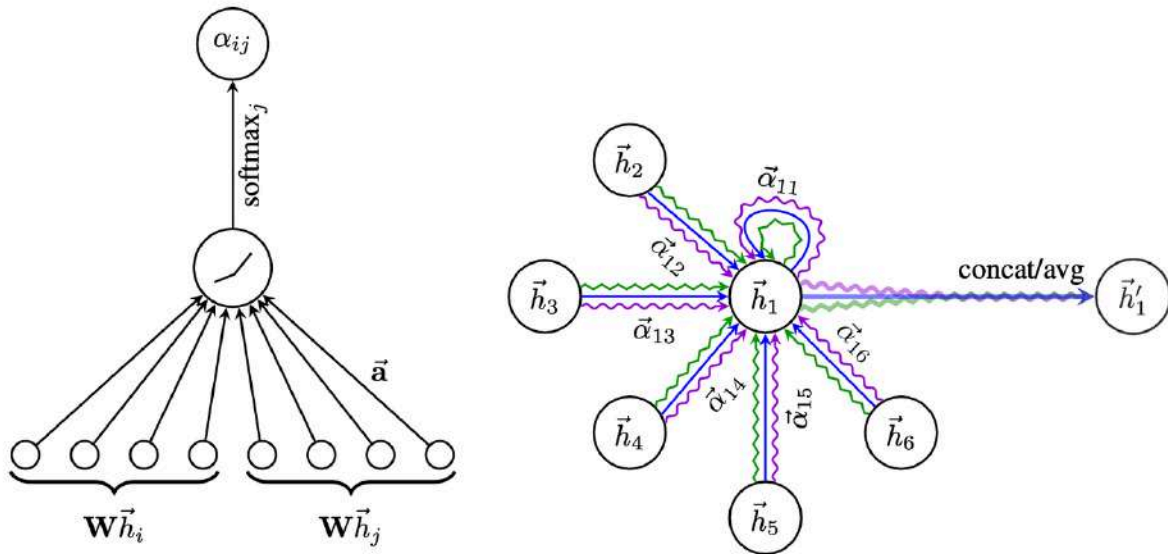


FIGURE 5.1 - GAT ATTENTION MECHANISM: ARROW THICKNESS = ATTENTION WEIGHT. NODES B AND C ARE MOST IMPORTANT; E IS MOSTLY IGNORED (A=0.05)

5.3 Working Process (Step-by-Step)

1. **Linear Transformation:** To increase the expressive power of the model, each node's initial features are multiplied by a shared learnable weight matrix (W). This projects the data into a higher-level feature space.
2. **Attention Coefficient Calculation:** The model calculates a raw attention score (e_{ij}) between a target node i and its neighbor j . This is achieved by concatenating their transformed features and passing them through a shared linear layer followed by a **LeakyReLU** activation.
3. **Normalization (Softmax):** To make the scores comparable across different nodes, the raw scores are passed through a Softmax function. This ensures that the final attention weights (α_{ij}) sum up to exactly 1.
4. **Weighted Aggregation:** Finally, the node calculates its new, updated feature representation (h'_i) by multiplying the transformed features of its neighbors by their respective attention weights and summing them up.



Left: The attention mechanism $a(W\vec{h}_i, W\vec{h}_j)$ employed by our model, parametrized by a weight vector $\vec{a} \in \mathbb{R}^{2F'}$, applying a LeakyReLU activation. **Right:** An illustration of multi-head attention (with $K = 3$ heads) by node 1 on its neighborhood. Different arrow styles and colors denote independent attention computations. The aggregated features from each head are concatenated or averaged to obtain $\vec{h}'_1$.

Figure 5.2 - GAT : Working Process

5.4 Simple Example

Imagine a Social Network where you (Node A) are trying to decide which movie to watch. You ask two connected friends (your neighbors):

- Friend B is a massive film buff who always gives great recommendations.
- Friend C barely watches movies and usually gives terrible advice.

In a standard GNN, your brain would average their advice 50/50. But your brain acts like a GAT. You dynamically assign an "attention weight" to them based on relevance. You assign 80% importance to Friend B and only 20% importance to Friend C. You aggregate their advice using these weights, leading to a much smarter final decision!

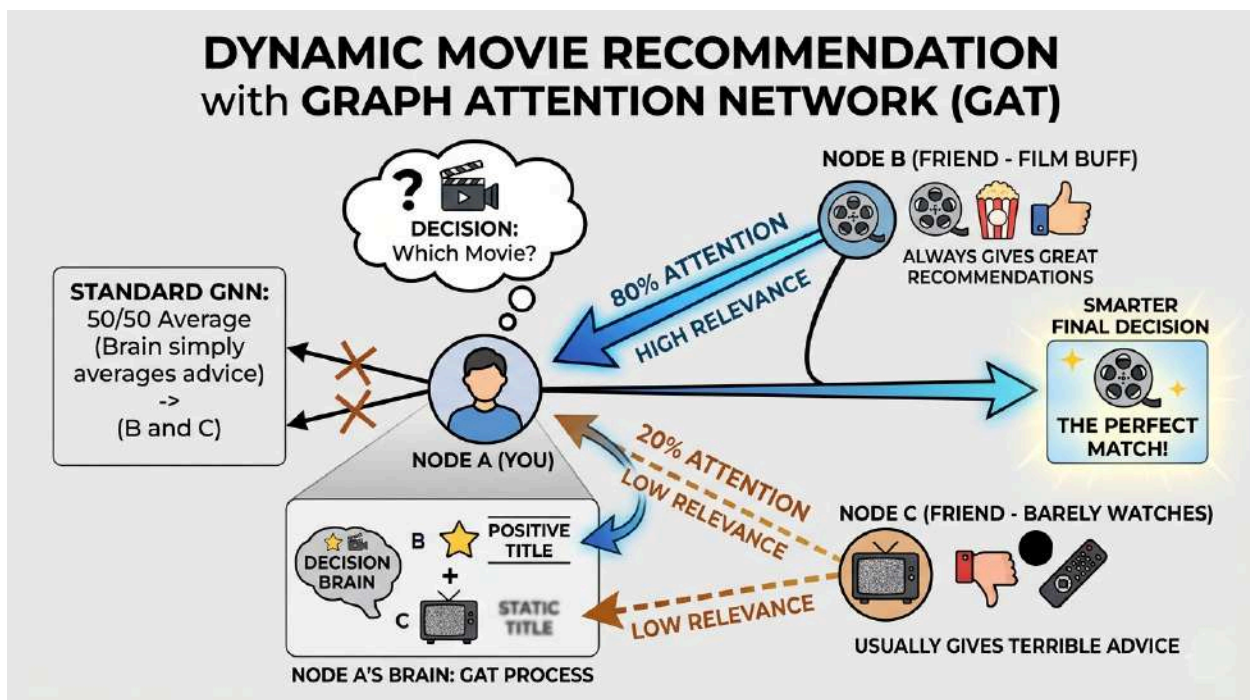


Figure 5.3 - GAT : Simple Example of Movie Recommendation

5.5 Mathematical Formulation of a GAT Layer

1. Linear Transformation

$$h'_i = Wh_i$$

2. Raw Attention Score

$$e_{ij} = \text{LeakyReLU}\left(a^T [Wh_i \parallel Wh_j]\right)$$

3. Normalized Attention Coefficient

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N_i} \exp(e_{ik})}$$

4. Final Node Representation

$$h'_i = \sigma\left(\sum_{j \in N_i} \alpha_{ij} Wh_j\right)$$

Where:

3. $h_i \in R^F$: Input feature vector of node i
4. $W \in R^{F' \times F}$: Learnable weight matrix (shared across all nodes)
5. $h'_i \in R^{F'}$: Transformed feature of node i
6. $a \in R^{2F'}$: Learnable attention weight vector
7. $[\cdot \parallel \cdot]$: Concatenation operation
8. e_{ij} : Raw attention score measuring importance of node j to node i
9. N_i : Set of neighbors of node i (including i itself)
10. α_{ij} : Normalized attention coefficient (importance weight)
11. σ : Nonlinear activation function (e.g., ReLU, ELU)

5.6 Mathematical Simulation

Let's simulate how Node A updates its features mathematically by paying attention to Node B and Node C.

The Scenario (3-Node Graph):

Node A is updating its data. The attention mechanism decides Node B is highly relevant ($\alpha_{AB} = 0.8$), and Node C is less relevant ($\alpha_{AC} = 0.2$).

- Transformed Features of Node B (Z_B) = [1.0, 2.0]
- Transformed Features of Node C (Z_C) = [3.0, 4.0]

The Math (Weighted Aggregation):

We multiply each neighbor's features by their calculated attention weight:

- **Node B Contribution:** $0.8 \times [1.0, 2.0] = [0.8, 1.6]$
- **Node C Contribution:** $0.2 \times [3.0, 4.0] = [0.6, 0.8]$

Final Aggregation (Sum):

$$h'_A = [0.8 + 0.6, 1.6 + 0.8] = [1.4, 2.4]$$

The new representation for Node A is [1.4, 2.4].

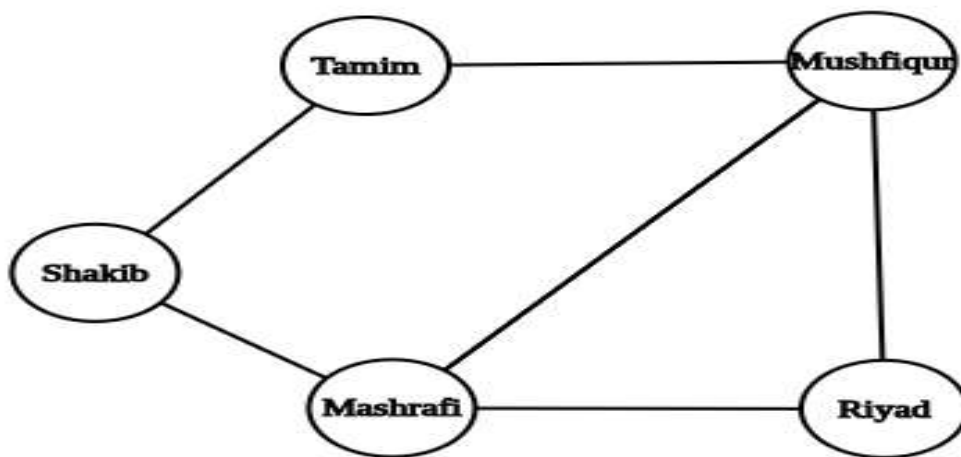
This simulation shows how a Graph Attention Network (GAT) updates a node's representation by giving different importance to its neighboring nodes. Since Node B has a higher attention weight (0.80) than Node C (0.20), Node B contributes more strongly to the final representation of Node A.

The final output $h'_A = [1.4, 2.4]$ is obtained through weighted aggregation of neighboring features. This demonstrates that GAT can focus more on relevant neighbors and less on less-important ones, allowing the network to learn more meaningful graph representations efficiently.

For Better Understanding :

Consider a cricket team scenario where five Bangladesh cricket players are represented as nodes in a graph. The goal of the model is to learn which teammates are most important when updating a player's representation.

In this graph, Shakib Al Hasan is connected with teammates such as Tamim Iqbal and Mashrafe Bin Mortaza. The GAT model learns which connected teammates are more important when updating Shakib's representation by assigning higher attention weights to more influential players.



Five players are modelled as nodes in a graph. An **edge** between two players means they play together frequently. Each player carries a **feature vector of 4 values**:

- Batting Average
- Balls Faced
- Fitness Score
- Matches Won



Example player data: Tamim Iqbal

Player Name	Batting Avg	Balls Faced	Fitness	Matches Won
Tamim Iqbal	38	30000	2	75

Tamim's row is highlighted (500 balls faced, fitness score 3, 25 wins). The complete feature matrix X has shape $[5 \times 4]$ — 5 players, 4 features each.

The raw features are projected into a higher-dimensional space using a learnable weight matrix W of shape $[4 \times 8]$.

Multiplying X (5×4) by W (4×8) gives H (5×8): each player now has **8 richer, transformed features** instead of 4. These embeddings are what the attention mechanism operates on.

Key point: The weight matrix W is shared across all nodes. It is learned during training and remains the same for every player — it defines the feature space in which attention scores are computed.

Attention Score e_{ij}

To decide how much **Shakib (S)** should attend to **Tamim (T)**, the model runs three sub-steps:

Concatenate

Stack Shakib's 8 transformed features and Tamim's 8 features side-by-side $\rightarrow$ vector $[1 \times 16]$.

Dot Product

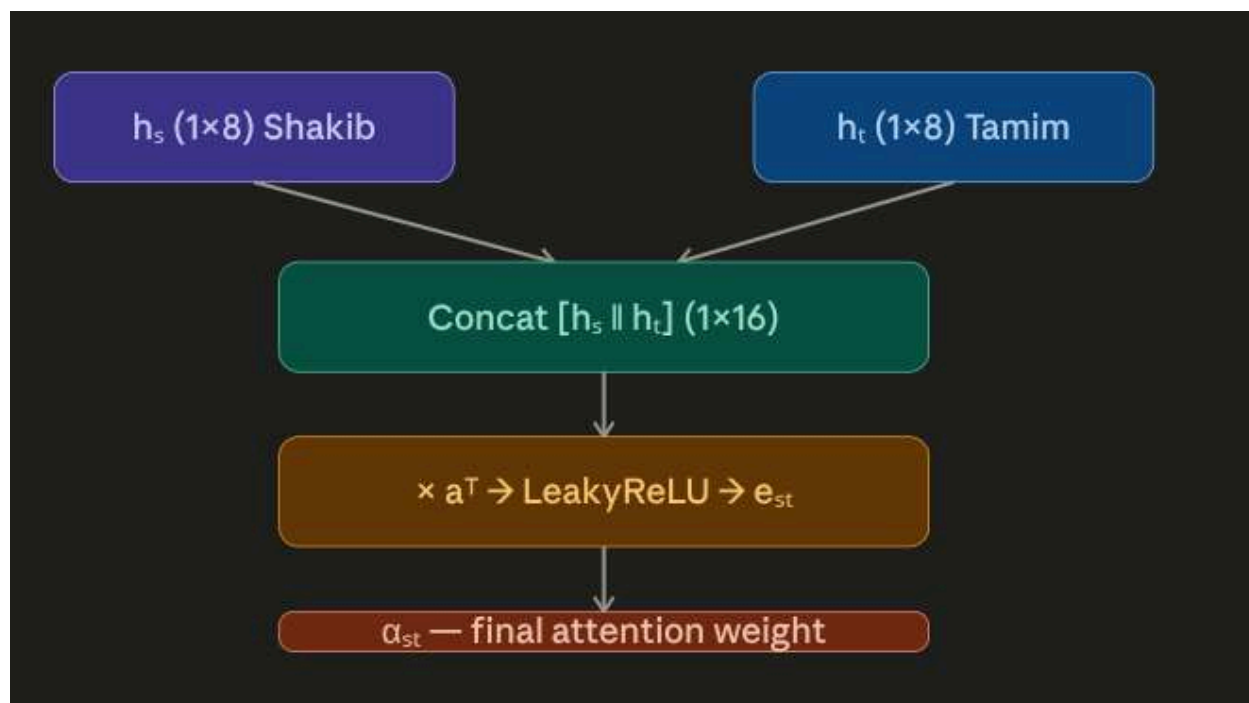
Multiply the 16-dim vector by a learnable attention vector a^T (also 1×16) $\rightarrow$ a single scalar.

LeakyReLU

Pass the scalar through a LeakyReLU activation to get the raw attention score e_{ST} .

Softmax Normalisation

The raw scores e_{ij} are normalised over all neighbours of node i using softmax, producing α_{ij} values that sum to 1 across each neighbourhood.



Attention scores outside the neighbourhood are **masked to $-\infty$** before softmax — so they effectively become zero after normalisation. This ensures players only aggregate information from their actual co-play partners.

The **adjacency matrix A [5 × 5]** records which players are neighbours. GAT only computes attention scores for pairs that share an edge.

	S	T	Msh	M	R
S	1	1	1	0	0
T	1	1	0	1	0
Msh	1	0	1	1	1
M	0	1	1	1	1
R	0	0	1	1	1

1 = connected, 0 = not connected (blue = active edge)

1 = connected, 0 = not connected. After applying the mask and softmax, the result is a **soft, weighted adjacency matrix** of the same shape $[5 \times 5]$ — the α matrix.

Step 5 — Aggregation (Final Embedding)

Using the attention weights, each node aggregates its neighbours' transformed features. For Shakib, the update is:

$$\mathbf{h}'_s = \sigma(\alpha_{ss} \cdot \mathbf{Wh}_s + \alpha_{sT} \cdot \mathbf{Wh}_T + \alpha_{s,Msh} \cdot \mathbf{Wh}_{Msh})$$

S (self) $\alpha = 0.5$

T (Tamim) $\alpha = 0.3$

Msh (Mash) $\alpha = 0.2$

These three weighted embeddings are summed, then passed through a non-linear activation σ to produce Shakib's **updated 8-dimensional representation $\mathbf{h}'_s$** .

The α values above are illustrative — in practice they are learned end-to-end from data. The **output matrix $\mathbf{H}'$** has shape $[5 \times 8]$ — one updated vector per player.

The benefit for Shakib Al Hasan is that his new representation no longer depends only on his own statistics. After GAT aggregation, his updated 8-dimensional feature vector also contains important information from influential teammates like Tamim Iqbal and Mashrafe Bin Mortaza.

For example:

- If Tamim is a strong batting partner, GAT may give him higher attention weight.
- If Mashrafe contributes leadership or match-winning experience, his information also influences Shakib's updated embedding.

So Shakib's final vector becomes more informative because it captures:

- his own performance,
- teammate relationships,
- team coordination patterns,
- and influential player interactions.

This helps the model make better predictions such as:

- player importance,
- captaincy decisions,
- match strategy,
- team recommendation,

- or performance analysis.

Without attention, every teammate would contribute equally. With GAT, the model automatically learns which teammates matter more for Shakib.

The power of GAT over a basic GNN lies in the **attention mechanism**:

Standard GNN: Averages all neighbours with equal weight every teammate contributes equally regardless of how similar or relevant they are.

This makes GAT **adaptive**; it **automatically** discovers which relationships matter most for the downstream task, without any manual feature engineering or hard-coded edge weights.

Multi-head attention (running K independent attention heads and concatenating the results) further improves stability and expressiveness, each head can specialise in a different type of relationship between players.

5.7 Pseudocode for GAT Layer

Below is the logical pseudocode demonstrating how a single Graph Attention layer processes node features:

Pseudocode

```
Function GAT_Layer(Node_Features, Graph_Edges):
    W = Learnable_Weight_Matrix
    a = Attention_Mechanism_Parameters

    // 1. Apply linear transformation to all node features
    z = Node_Features * W

    For each Node i in Graph:
        Raw_Scores = empty_list()
        For each Neighbor j of Node i:
            // 2. Calculate raw attention score between Node i and Neighbor j
            e_ij = LeakyReLU(a_transpose * Concatenate(z_i, z_j))
            Raw_Scores.append(e_ij)

            // 3. Normalize the raw scores to get attention weights (summing to 1)
            Alpha_i = Softmax(Raw_Scores)

            // 4. Calculate the new feature representation using weighted aggregation
            h_prime_i = ELU(Sum_over_all_j(Alpha_ij * z_j))

    Return h_prime (The updated features for all nodes)
```

5.8 Python Implementation

Note: All practical implementations for this assignment are available in my Google Colab notebook. [Click Here](#)

5.9 Which Problems It Solves & How It Resolves Other Limitations

Traditional Spectral Graph Convolutions (like GCNs) require massive, costly mathematical operations (like matrix inversions) over the *entire* graph structure at once. Furthermore, they are **Transductive**, meaning they only work on the exact graph they were trained on. GAT solves this by using local, shared attention layers. Because the attention mechanism is learned locally between pairs of nodes, GATs are **Inductive**—they can be trained on one graph and perfectly applied to a completely new, unseen graph.

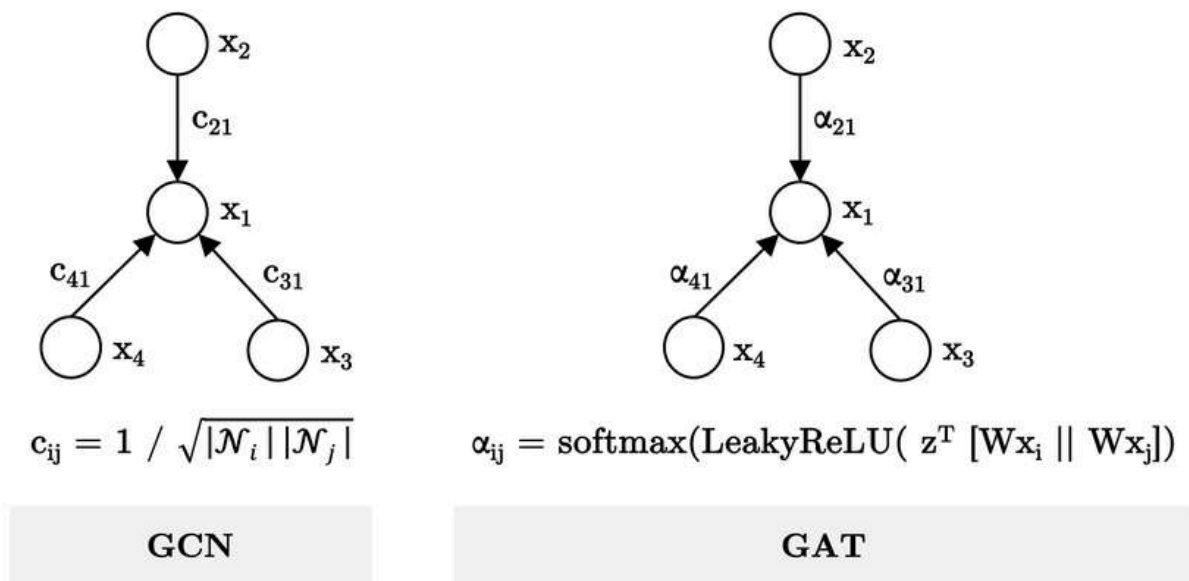


Figure 5.4 - GAT : How GAT Resolves Limitations of GCN

5.10 Advantages

1. **Parallelization:** Because the attention calculation only happens locally between connected nodes, operations are highly parallelizable across all nodes and edges, making it computationally efficient.
2. **Interpretability:** Unlike "black box" algorithms, researchers can inspect the final attention weights (🔗) to understand *exactly* which neighbors drove the model's final predictions.

3. **Multi-Head Attention:** GATs can use multiple "heads" (learning different attention weights simultaneously) to capture different relationships (e.g., in a social network, one head pays attention to family ties, another to workplace ties).

5.11 Challenges & Limitations

1. **Static Attention:** The original GAT can suffer from "static attention"—meaning the ranking of a neighbor stays the exact same regardless of what specific query the central node is asking. (This was recently fixed in a newer version called **GATv2**).
2. **Oversmoothing:** Reaching distant nodes requires stacking many layers. If a GAT is too deep, the features of all nodes blend together until they look identical, destroying the model's accuracy.

5.12 Real-World Applications

1. **Citation Networks (e.g., Cora Dataset):** Predicting the specific topic of a scientific paper based solely on the network of other papers it cites, dynamically weighing highly influential papers over minor ones.

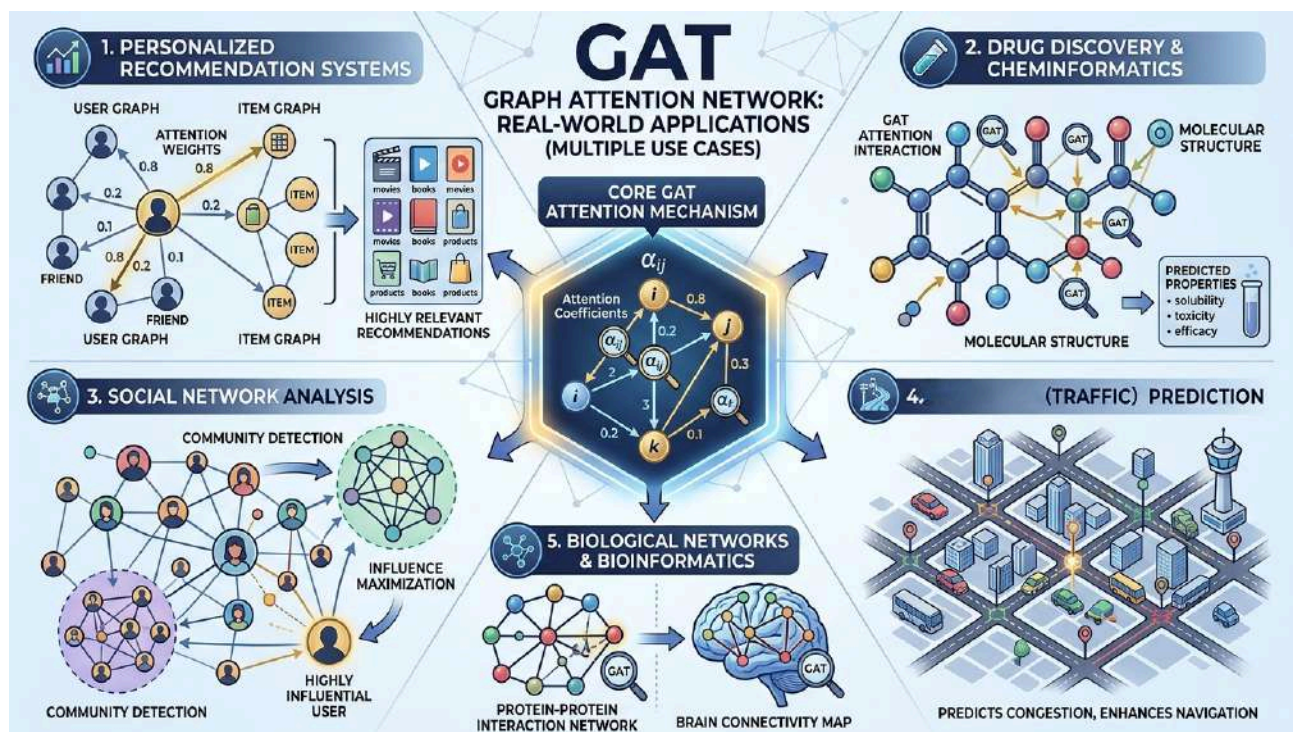


Figure 5.5 - GAT : Real World Applications

2. **Bioinformatics:** Analyzing Protein-Protein Interaction (PPI) networks. Proteins interact in complex ways; GATs can predict a protein's cellular function by focusing its attention only on the most critical biological neighbors.
3. **Traffic Forecasting:** Modeling a city map as a graph where intersections are nodes and roads are edges. The model can predict future traffic jams by heavily weighting connected intersections that are currently experiencing crashes or rush-hour flow.

5.13 Future Directions and Open Problems

1. **Dynamic Attention (GATv2):** Continued research into completely dynamic attention where the importance of a neighbor changes depending entirely on the target node's immediate context.
2. **Heterogeneous Graphs:** Standard GATs assume all nodes and edges are the exact same "type". Future models are expanding to effectively handle massive heterogeneous graphs (e.g., an e-commerce graph containing "User" nodes, "Product" nodes, and "Review" nodes, all interacting differently).
3. **Solving Oversmoothing:** Inventing new architectural skip-connections or normalization techniques that allow GATs to be stacked 50 to 100 layers deep without all the node features washing out into a single uniform average.

5.14 References for Further Reading

01. Velickovic, P., et al. (2018). "Graph Attention Networks." ICLR. (The foundational GAT paper)
02. Brody, S., et al. (2022). "How Attentive are Graph Attention Networks?" ICLR. (Proposes GATv2 — fixes static attention limitation)
03. Hamilton, W. L. (2020). "Graph Representation Learning." Morgan & Claypool Publishers.

— End of Document —

Advanced Machine Learning Study Material · 5 Topics · FL · AFF · Octree · DWT · GAT